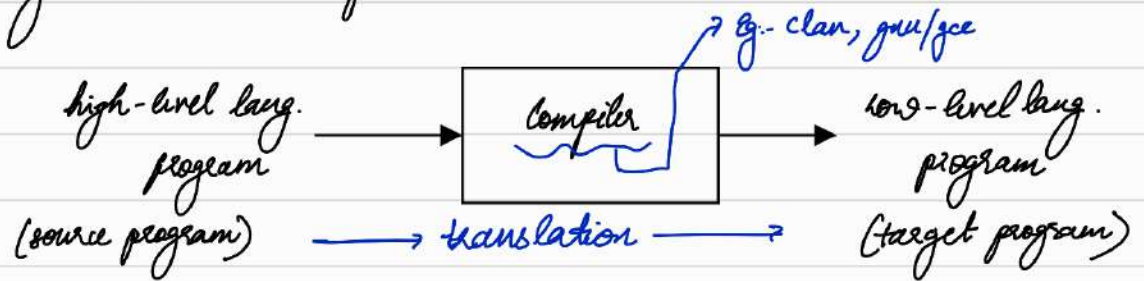


Overview:

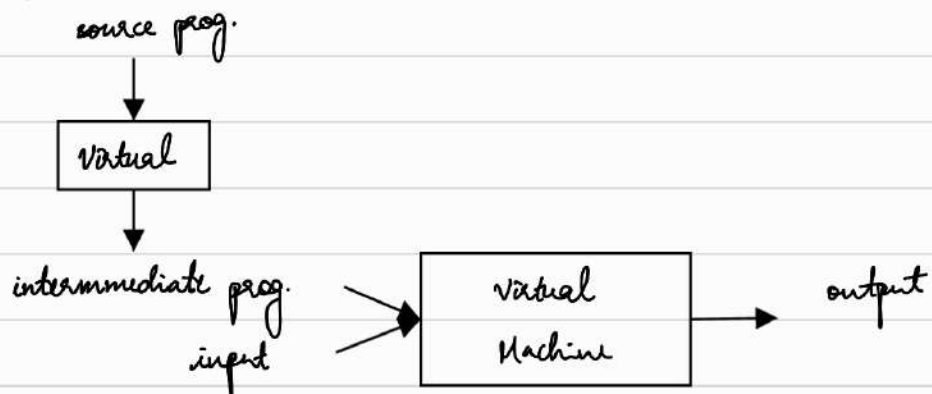
- Considering Black Box as a compiler



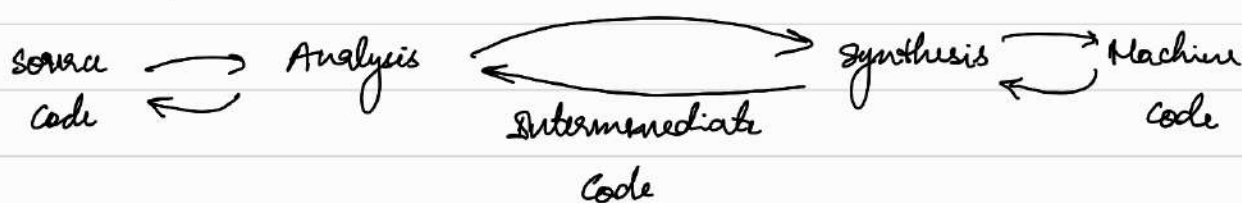
- task: detects errors & optimises performance
- should have an idea :-
 - regular expressions
 - pushed-down automata
 - DFA & NFA's
 - how to use GDB's for debugging

Compiler	Interpreter
→ checks the whole code	→ line by line
→ more time o/p	→ less time o/p
→ error detection difficult	→ Easier debugging
→ obj. file created	→ no obj. code created
→ more memory req.	→ less memory requirement

- Hybrid Compiler



• Structure of Compiler



→ Modules :-

- ↓ source code
- Phases of compiler :-
- ① Lexical Analyzer (M1)
 - ② Syntax Analyzer (M2)
 - ③ Semantic Analyzer (M3)
 - ④ Intermediate Code generator (M4)
 - ⑤ Machine Independent Code Optimizer (M5)
 - ⑥ Code generator (M6)
 - ⑦ Machine Dependent Code Optimizer (M7)
 - ⑧ Symbol Table (M8) [Data struct. that stores all identifiers]

M1: Introduction

Grammar : set of rules/instructions

Eg:- for a C program grammar,

→ datatype, identifiers,

→ DT id, id, id

should be represented as

letter [letter/no./_]* (Regular Expression)

→ consists of production side which has variables that are non-terminals that appear of left side of prod. side, and right side: terminals with rules in prod. rule like

non-terminals { A → BC/a } terminals { B → b, C → c }
LHS of prod rule is start symbol of grammar

lexical Analysis

→ list of regular Expressions

→ aka "scanning"

→ reads char. from source prog. and grps them into chunks called lexeme.

Ex:- `pes = initial + rate * 60` → Lexical Analyzer [checks grammars]

* language :- set of terminals

:- non-terminals imposes a kind of hierarchy.

6/1/25
Semantic Errors

5/1/25

Lab Instructions: ① C knowledge, GDB knowledge

② MS Teams Assignment (DOCS, ODD file) Submission

→ Name, reg no., section, title of Lab

→ Program

→ snapshot i/p & o/p

reg no., path

6/1/25

Q)
A:

string = id + id * id

$E \Rightarrow E * E$

$E \rightarrow E + E$
$E \rightarrow E * E$
$E \rightarrow id$

$\Rightarrow E + E * E$

$\Rightarrow id + E * E$

$\Rightarrow id + id * E$

$\Rightarrow id + id * id$

$E \rightarrow E + E / E * E / id \Rightarrow$

$E \Rightarrow E + E$

$\Rightarrow id + E * E$

$\Rightarrow id + id * E$

$\Rightarrow id + id * id$

has more than
a leftmost
derivation
↳ means
Ambiguous
grammar

Q)

string: id + id * id

expression grammar $\Rightarrow E \rightarrow E + T / T$

$T \rightarrow T * F / F$

$F \rightarrow id$

A:

$E \Rightarrow E + T$

tried
in
class
(wrong
answer)

~~$T + T$~~

~~$T * F + T$~~

~~$T * id + T$~~

~~$F * id + T$~~

~~$id * id + T$~~

~~$id * id + F$~~

~~$id * id + id$~~

$F + T$

$id + T$

$id + T * F$

$id + F * F$

$id + id * F$

$id + id * id$

correct leftmost
derivation solution

→

string which matches w. source program → lexeme

→

lexical Analyzer generates tokens for each lexeme

syntax for tokens:-

<token_name, attribute values>

terminal names

6/1/25

lexical analyzer does not generate token for space.

Ex :- Considering, $pos = initial + rate * 60$

lexeme	token		Symbol Table
pos	$\langle id, 1 \rangle$		lexeme pos
=	$\langle = \rangle$	1	Type
initial	$\langle id, 2 \rangle$	2	initial
+	$\langle + \rangle$	3	rate
rate	$\langle id, 3 \rangle$		
*	$\langle * \rangle$		
60	$\langle number \rangle$		

9/1/26

$pos = initial + rate * 60$ character stream

Lexical Analyzer

lexeme generating tokens

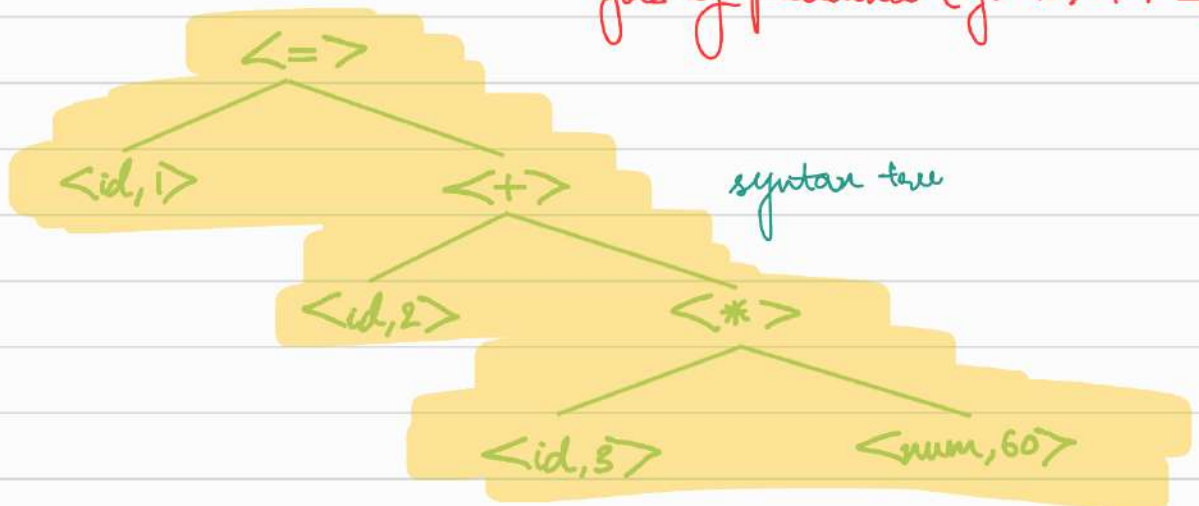
$\langle id, 1 \rangle \langle = \rangle \langle id, 2 \rangle \langle + \rangle \langle id, 3 \rangle \langle * \rangle \langle num, 60 \rangle$ token stream

Syntax Analyzer

Parser

Syntax tree :- operators (interior nodes), operands (leaf nodes)

goes by precedence (Eg:- $* > + > =$)



syntax tree

syntax tree (i/p)

Semantic Analyzer

Tokens are not generated for blank spaces

Assuming,

	Lexeme	Type
1	pos	float
2	initial	float
3	rate	float

It follows the logical rules of the programming language.



Syntax tree + symbol table

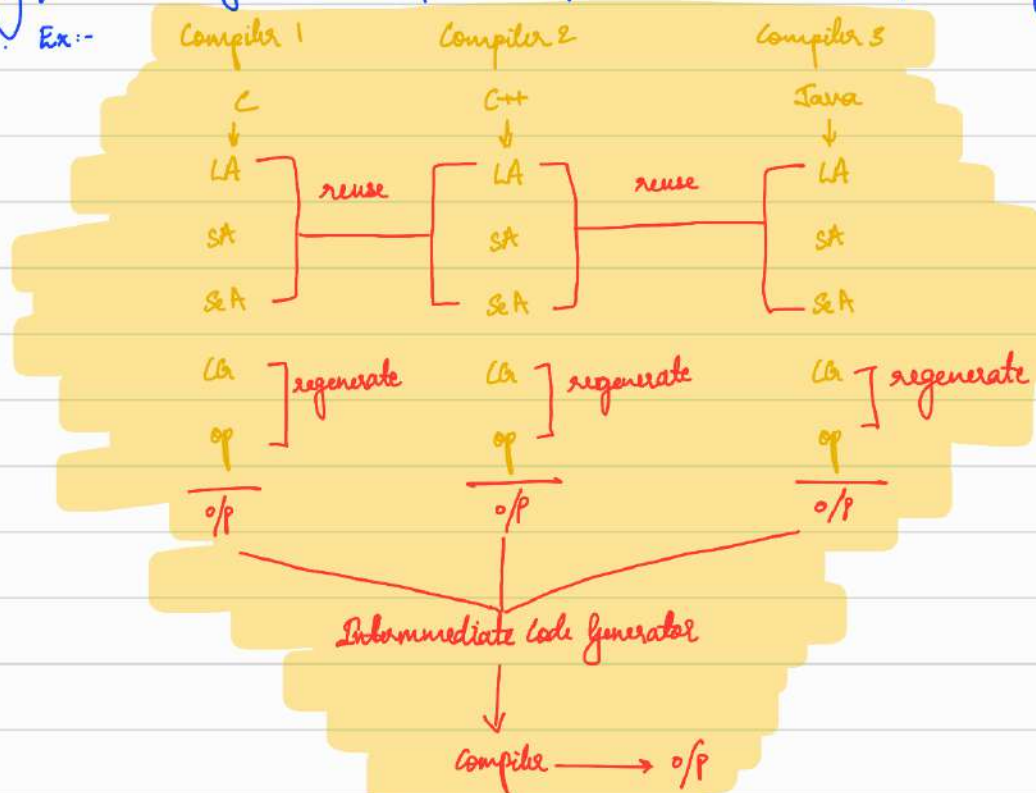
- Role :-
- ① Type checking
 - ② Function call Validation
 - ③ Symbol table Management
 - ④ Scope resolution
 - ⑤ Syntax tree Annotation.

int to float

<num, 60.0>

Intermediate Code Generator

rewriting phases & code for each compiler is expensive in cost & computationally intensive. Ex:-



for reusing we regenerate only @ Code generator & optimization whilst maintaining
 $LA \rightarrow SA \rightarrow se A$ into a repeated block, and at the end all 3 compilers
are put into an intermediate block which will improve characteristics
below,

- ① Easy to produce
 - ② Easy to translate
- Improves speed lowering cost

This is achieved using

Three Address Code

→ intermediate

→ contains assembly like instructions w. atmost 3 operands
& atmost one operator on RHS.

eg:- $t1 = \text{inttofloat}(60)$
 $t2 = id3 * t1$
 $t3 = id2 + t2$
 $id1 = t3$

temporary variable

from syntax

intermediate representation

Machine Independent Code Optimization

$t1 = id3 * 60.0$
 $id1 = id2 + t1$

basically reducing no. of variables

→ intermediate representation

Code Generator

basically converts machine to Assembly Code

LDF R1, id3
MULF R1, R1, #60.0
LDF R2, id2
ADDF R2, R2, R1
STF id1, R2

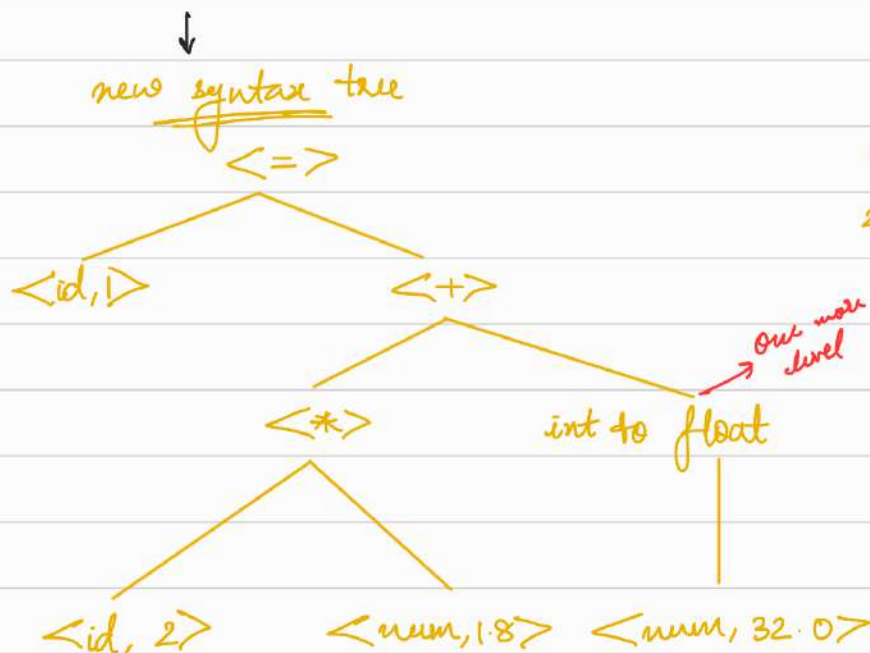
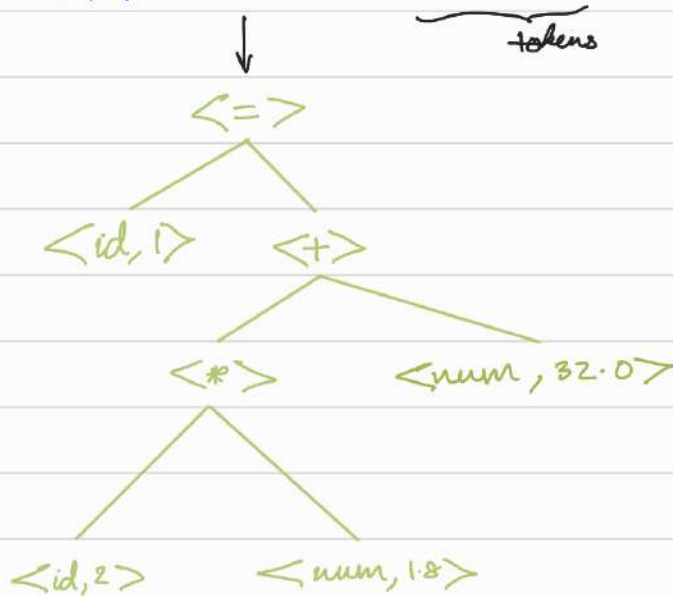
→ target machine code

Ex 2:

$F = C * 1.8 + 32$ (Question)

A:

$\langle id, 1 \rangle \Rightarrow \langle id, 2 \rangle \langle * \rangle \langle num, 1.8 \rangle \langle + \rangle \langle num, 32 \rangle$



	hexcode	Type
1	F	float
2	C	float

$t1 = id2 * 1.8$

$t2 = \text{int to float } (32.0)$

$t3 = t1 + t2$

$id1 = t3$

↓

$t1 = id2 * 1.8$

$id1 = 32.0 + t1$

↓

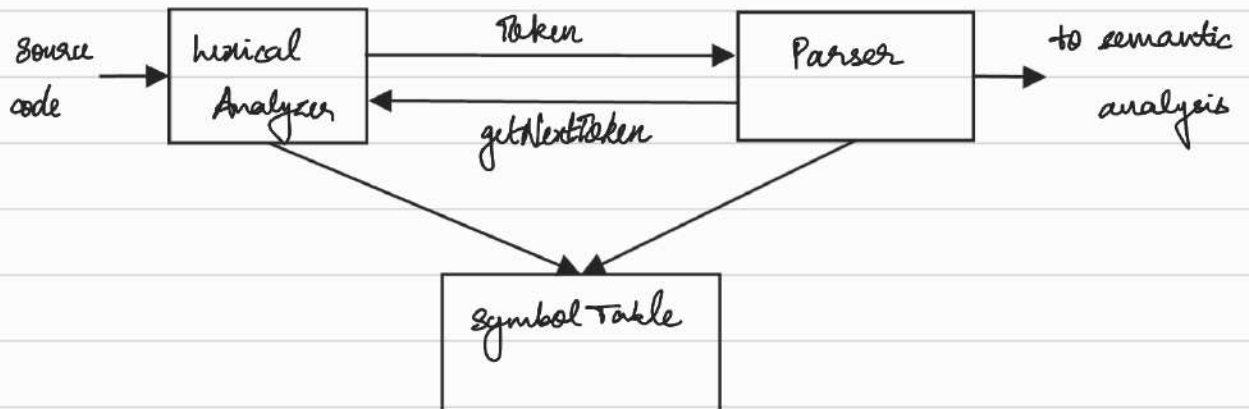
```

LDF    R1, id2
MULF   R1, R1, #1.8
ADDF   R1, R1, #32.0
STF    id1, R1
  
```

12/1/26

NOTE:- Instructions travel Right to left from bottom if on same level, diff. level (left to right).

M2: lexical Analysis



NOTE: for single line/multi line comments No token is generated.

→ token

- a pair of token name & optional attribute value.
- an abstract symbol representing a kind of lexical unit.
- eg:- a particular keyword, or a seq. of i/p characters denoting an identifier.
- token names are the i/p symbols that the parser processes.
- we will often refer to a token by its token name.

→ pattern

- description of the form that lexeme of a token may take.
- case: keyword as a token, the pattern is just the seq. of characters that form the keyword.
- case: identifiers & some other tokens, the pattern is a more complex structure that is matched by strings.

→ lexeme

- seq. of characters in the source prog. that matches the pattern for a token & is identified by the lexical analyzer as an instance of that token.

Token's	Problems
if	i followed by f
else	e followed by l followed by s followed by e
comparison	>, <, =, >=, <=, !=
id	$[a-zA-Z][a-zA-Z0-9_]*$ → underscore
num	$[0-9]^+$
literal	anything inside " "

Q) Write token names for `printf ("Total = %d\n", score);`

A:

$\langle id, 1 \rangle \langle C \rangle \langle string\ literal \rangle \langle , \rangle \langle id, 2 \rangle \langle \rangle \langle ; \rangle$

Tokens	Informal Description	Sample lexemes
if	characters i, f	if
else	characters e, l, s, e	else
comparison	<, >, <=, >=, ==, !=	<=, !=
id	letter followed by letters & digits	pi, score, D2
number	any numeric constants	3.14159, 0, 6.02e23
literal	anything surrounded by " "s	"core dumped"

Q) Write token names & associated values for the statement,

$$E = M * C ** 2$$

A:

$\langle id, E \rangle \langle assign_op \rangle \langle id, M \rangle \langle mult_op \rangle \langle id, C \rangle \langle exp_op \rangle \langle number, 2 \rangle$

$\langle id, E \rangle \langle = \rangle \langle id, M \rangle \langle * \rangle \langle id, C \rangle \langle ** \rangle \langle num, 2 \rangle$

Both are fine for exam

Lexical Errors

Hint :- seq. char. from i/p file don't match any patterns

Eg:- int 2/10; → any other language (emoji's still get solved by using ';')

Recovery :- ① Reports lexical Error
② Recovery mode: Panic mode

strategy 1: Deleting characters

int 2/10;
↓ ↓ ↓ ↓ ↓
<int> × × × × ×
delete

strategies
for recovering
lexical errors

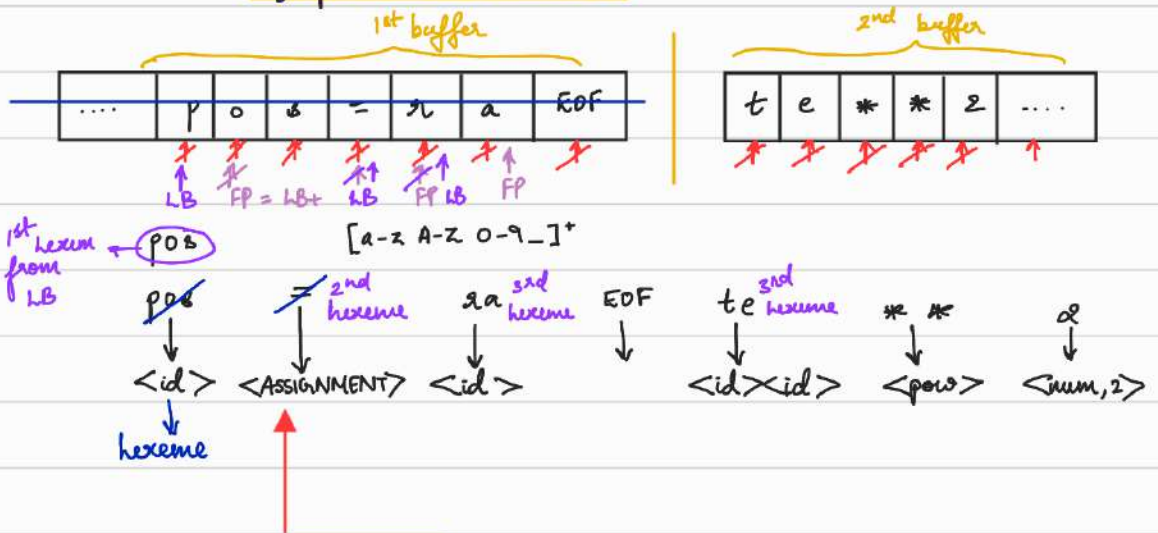
strategy 2: Inserting missing char. to remaining i/p.

strategy 3: Replacing a char. by another char.

strategy 4: Transpose adjacent 2 characters.

Input buffering

Q) pos = rate ** 2



Lexical Analyzer needs to look ahead to verify b/w '=' or '==' before being ruled to

Two pointers are used to handle buffers :- ① hexmebegin

② Forward (scans until pattern match is formed)

→

Specifications of Tokens

→ Regular definitions

① Identifiers

letter $[a-z A-Z _]$

digit $[0-9]$

identifier $\text{letter} - (\text{letter} _ / \text{digit})^*$

Q) Write regular definition for unsigned number integer or floating.

Sample Case :- 5280, 0.01234, 6.236E4, 1.89E-4, 5E5

A:

digit $\rightarrow [0-9]$

digits $\rightarrow \text{digit} \cdot \text{digit}^*$

optional fraction $\rightarrow (\cdot \text{digits}) / \epsilon$
(float)

optional exponent $\rightarrow (E (+|-| \epsilon) \text{digits}) / \epsilon$

number $\rightarrow \text{digits optional fraction optional exponent} / \epsilon$

$\lambda / \epsilon \rightarrow$ empty string

$- + \rightarrow$ signs

15/1/26

→

Recognition of tokens

stmt $\rightarrow$ if expr then stmt

| if expr then stmt else stmt

| other

expr $\rightarrow$ term relop term | term

term $\rightarrow$ id | number

Non Terminal $\rightarrow$ stmt

Terminals $\rightarrow$ if, expr, then, else, other

Keywords $\rightarrow$ expr, other

→

Terminal and Non-Terminals

$A \rightarrow BC | a$

$B \rightarrow b$

$C \rightarrow c$

Non-Terminal (NT) $\rightarrow A, B, C$

terminal (T) $\rightarrow a, b, c$

if $\rightarrow$ if
else $\rightarrow$ else

then $\rightarrow$ then

relop $\rightarrow = / < > / < / > / > = / < =$

digit $\rightarrow [0-9]$

letter $\rightarrow [a-z A-Z]$

digits $\rightarrow [0-9]^+$

id $\rightarrow \text{letter} (\text{letter} / \text{digit})^*$

NOTE:- 'X' is ϵ meaning "NVhh"

'+' $\rightarrow$ one or more occurrence

'?' $\rightarrow$ zero or one occurrence

'*' $\rightarrow$ zero or more occurrence

'.' $\rightarrow$ except \n, single char match

Ex:

num $\rightarrow$ digits ($\cdot$ digits)? (ϵ [+ -] digits)?

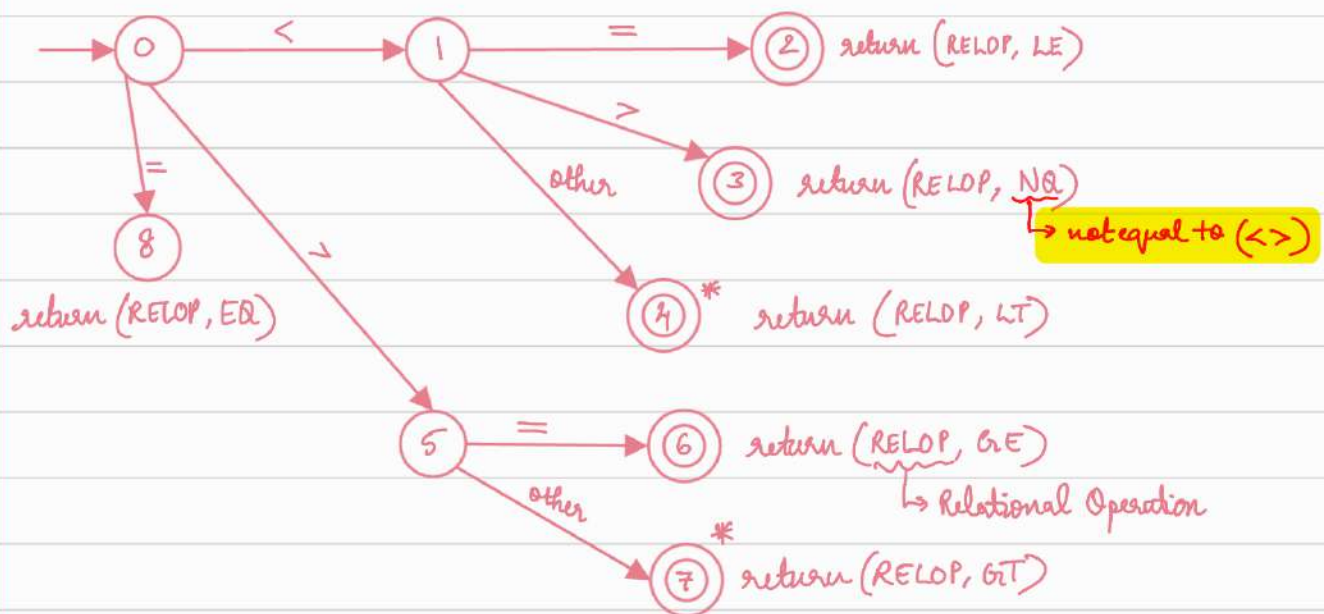
A:

Transition Diagram

NOTE:- final state must return a token

for relop

relop $\rightarrow$ = / < / <= / < / > / >=



NOTE:- Retract is shown by star (*)

yylex()
yywrap() → eof

yytext — text matched

yyklen — no. of char

yyval — value of curr. token

yyin → points to 1st file (FP)

sections :- ① declaration

② rules

→ Execution steps

→ \$ flex filename.l (\$ flex count.l)

→ \$ gcc lex.yy.c -o output

→ \$./output

→ Transition Diagram for unsigned number

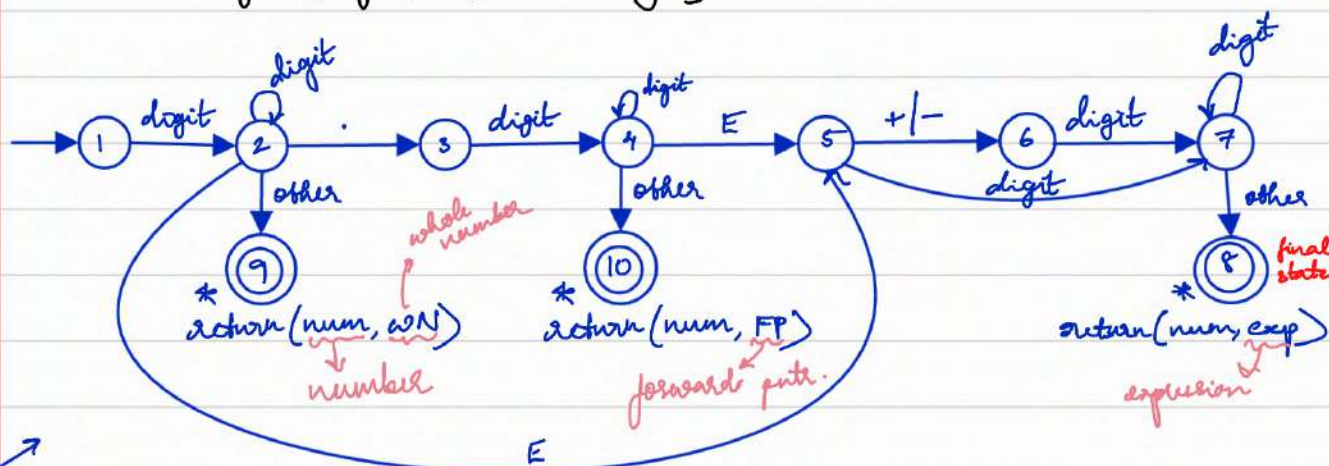
Q) Draw the transition diagram for the below regular definition.

digit $\rightarrow [0-9]$

digits $\rightarrow \text{digit}^+$

number $\rightarrow \text{digits} (\cdot \text{digits})? (E [+ -]? \text{digits})?$

} regular definition



Q) Sample Case :- 5280, 0.01234, 6.236E4, 1.89E-4, 5E5 to verify the above (↑) transition diagram

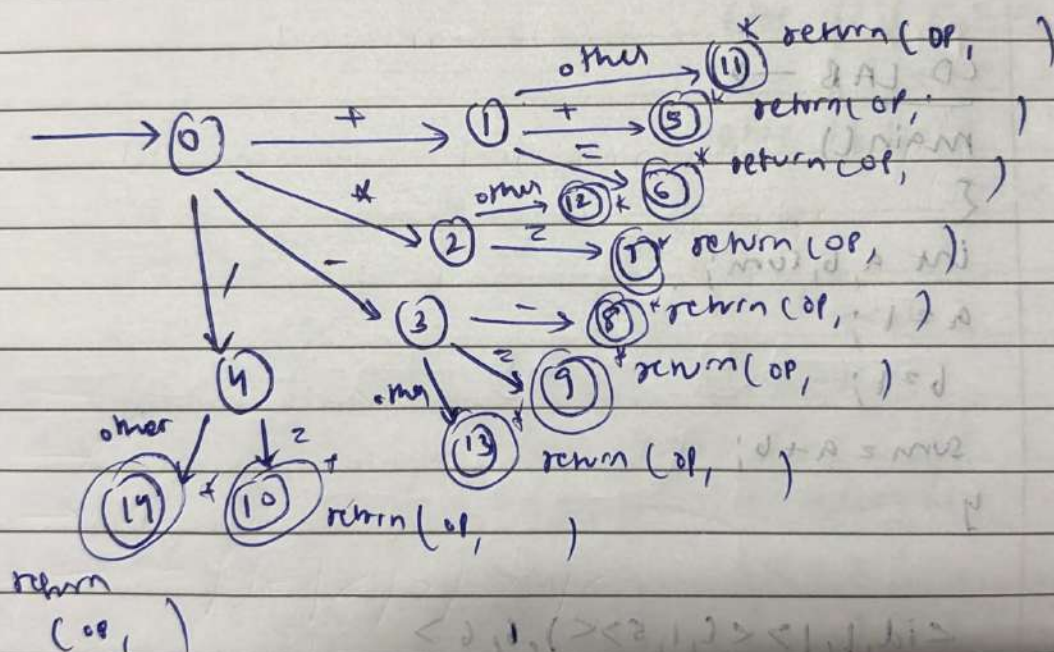
A: the above transition diagram is valid and accepts the following set of no.'s.

Q) Draw transition diagram for given regular definition,

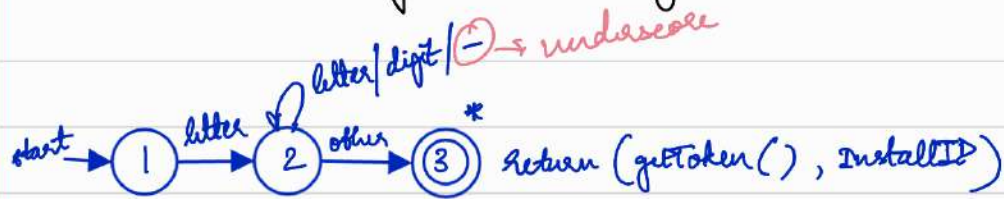
o/p $\rightarrow + | * | - | / | ++ | -- | + = | - = | / = | * =$

$\Rightarrow$ regular definition

A:



→ Transition Diagram for id's & keywords



NOTE: Keywords are pre-installed in the symbol table from where InstallID is called for the lexeme, and getToken()

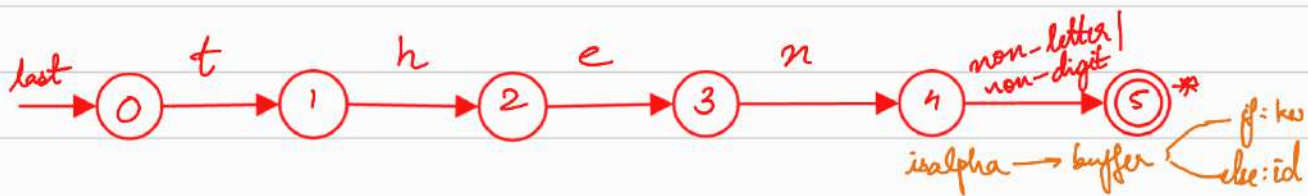
gets placed by its InstallID.

Another method is,

handle_kws

⇒

lexeme	InstallID
int	kws
a	id
for	kws



20/1/20

index -1 ; type (NULL)

lab3

<id, line 1> <1> <

<main, 1, 1> <C, 1, 5> <>, 1, 6> <1, 2, 1>

<int, 3, 1> <id, 3, 5> <,, 3, 6> <id, 3, 7> <,, 3, 8> <id, 3, 9>
<,, 3, 10>

<id, 4, 1> <=, 4, 3> <1, 4, 5> <,, 4, 6>

<id, 5, 1> <=, 5, 3> <1, 5, 5> <,, 5, 6>

<id, 6, 1> <=, 6, 5> <id, 6, 7> <+, 6, 9> <id, 6, 11> <,, 6, 12>
<3, 7, 1>

isalpha()

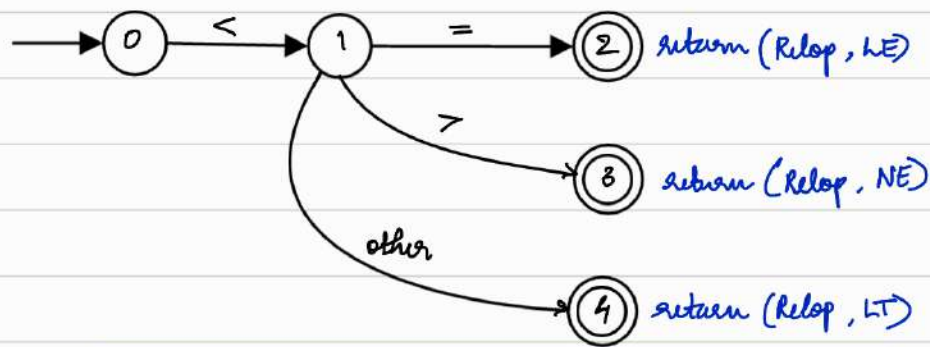
c = fgetc(fp)

if (c == '+' | '-' | '#' | '/')

tk.tokenname = c

return tk

22/1/26



* not written all states, as it was written before

Architecture of a transition Diagram based lexical Analyzer

Token getRelop() {

Token retToken = new (RELOP);

while (1) {

switch (state) {

case 0:

c = nextchar();

if (c == '<')

state = 1;

else if (c == '>')

state = 5;

else if (c == '=')

state = 8;

else

fail();

break;

case 1: c = nextchar();

if (c == '=')

state = 2;

else if (c == '>')

state = 8;

else

state = 4;

break;

* case 2 & 3 should write ourselves as it consists of only return

case 2:

```
return ();  
return.attribute = "LE"  
return(outToken);  
break;
```

case 3:

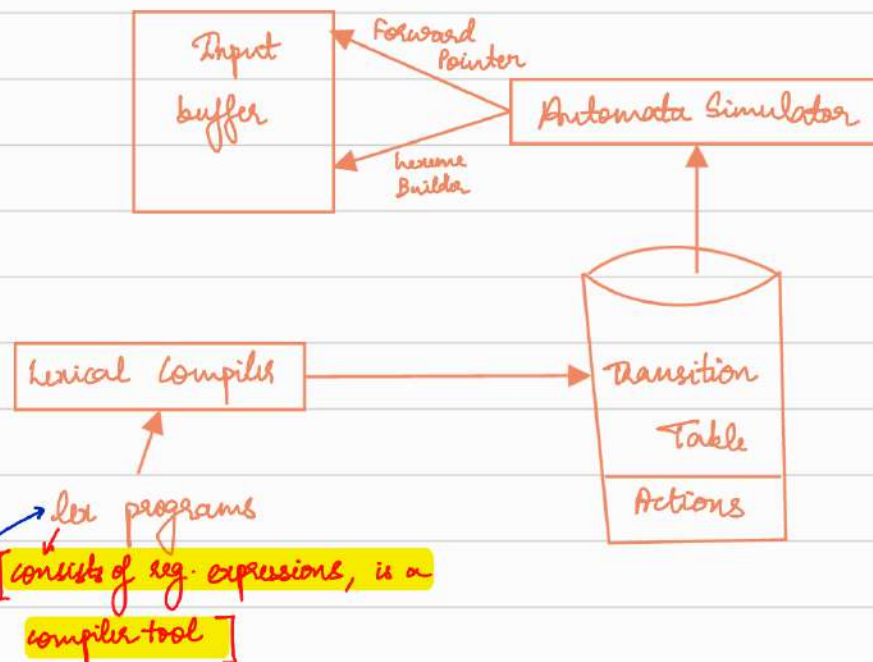
```
return ();  
return.attribute = "LE"  
return(outToken);  
break;
```

case 4:

```
return ();  
return.attribute = "LE"  
return(outToken);  
break;
```

... (continue to complete as per state transition diag. drawn earlier)

Structure of the generated analyzer - Finite Automata



more on this
would be discussed
in lab

Lexical Program Structure

Reg. exp

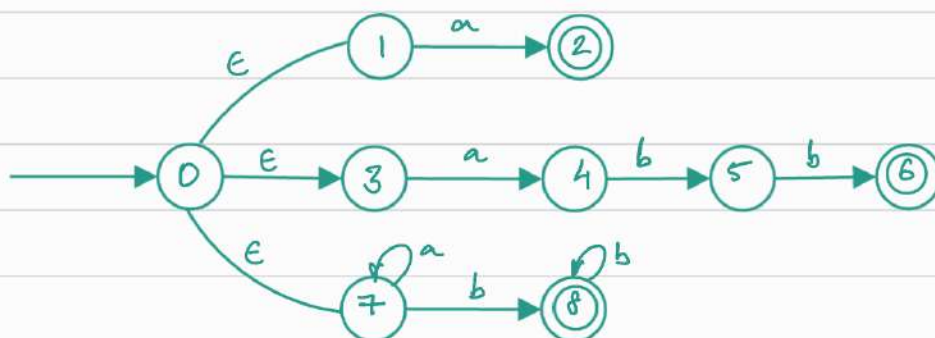
actions

Pattern matching Based on NFA's

Ex: We have 3 patterns

p_1	a	$\{A_1\}$
p_2	abb	$\{A_2\}$
p_3	a^*b^+	$\{A_3\}$

NFA



~ combined NFA

→ of the start symbol

Closure (start state of new combined NFA)

Closure {states} = $\{0, 1, 3, 7\}$

Suppose, Input = abb

then, $abb \Rightarrow p_2$ & p_3 are matched

Input = $\frac{a}{\downarrow} \frac{bb}{\downarrow}$
considered substring

then, $\Rightarrow (p_1 p_3)$ is matched

$\downarrow$
 p_1 followed by p_3

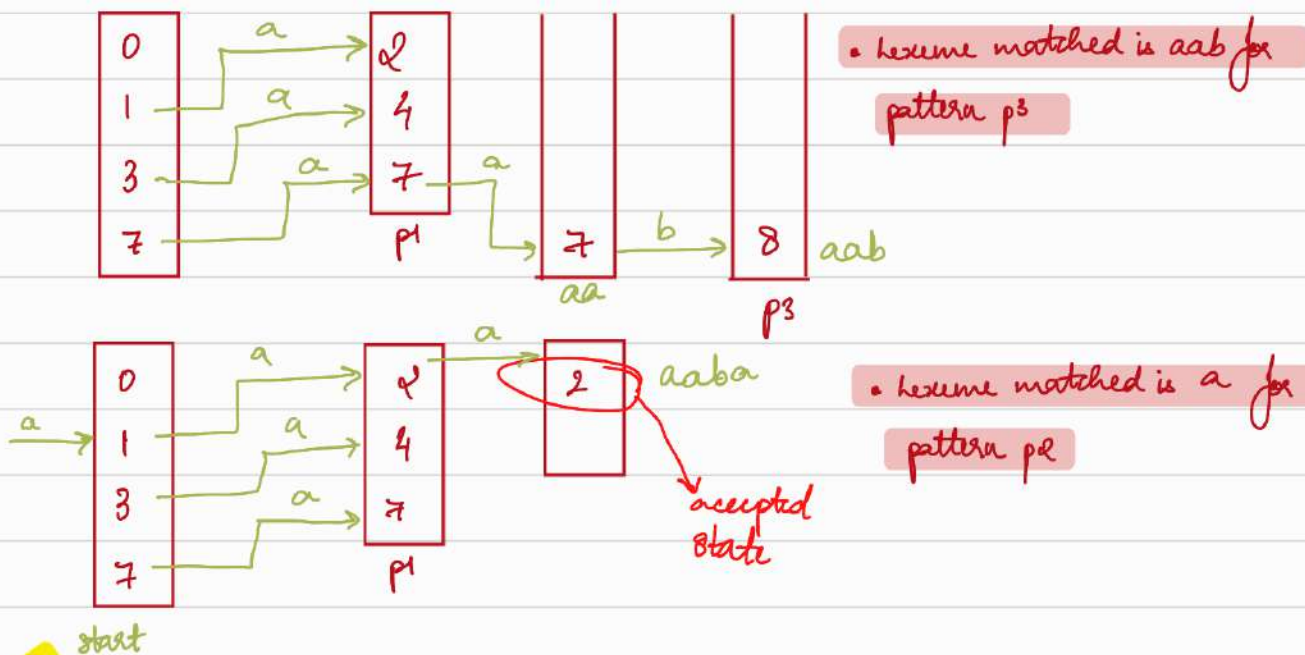
$\Rightarrow$ Conflict resolutions of lex programs (rules)

① takes longest matching pattern.

as per Input = $a \quad bb \Rightarrow p_2$ is matched

Input = $aaabbb \Rightarrow p_3$ is matched

Input: ~~aa~~ba



• lexeme matched is aab for pattern p3

• lexeme matched is a for pattern p2

String ch. of compilers

M4: Syntax Analysis

[Used push down automata]

Parser

- ↳ Tokens + Grammars (i/p to Syntax analyzer)
- ↳ Parser Tree

Eg:- main() {
 int a, b;
 a = 5;
}

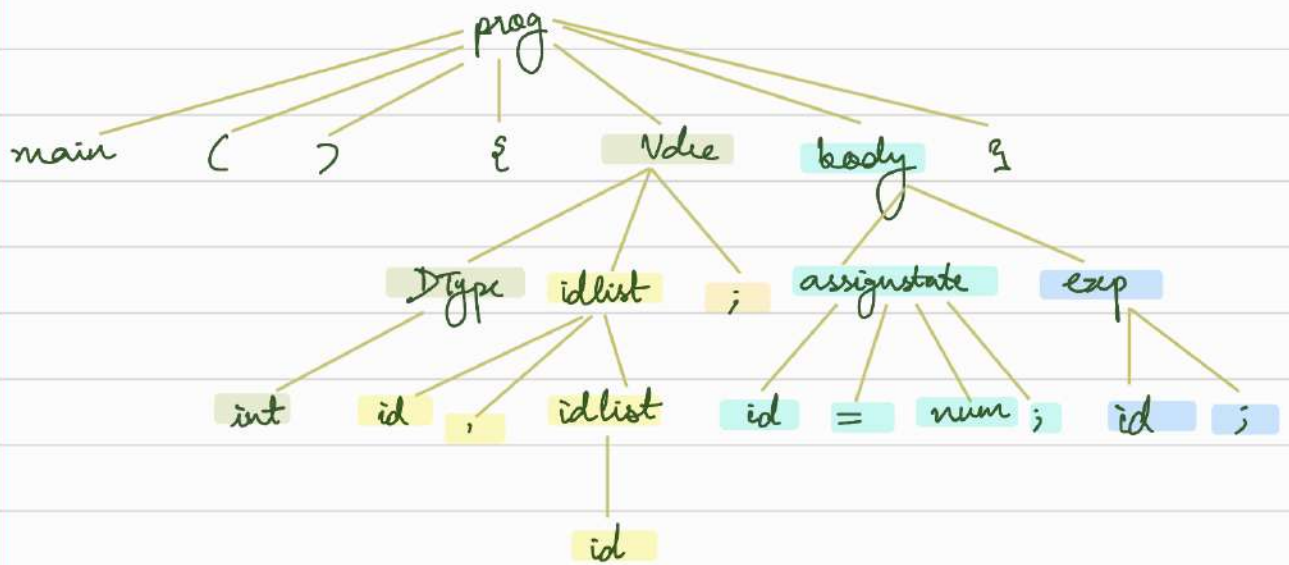
program

prog → main() { Vdec body }
Vdec → DType idlist;
DType → int/float
idlist → id/id, idlist
body → assignstat/exp
assignstat → id = num;
exp → id;

grammar

easier to translate to grammars and decide errors

Parse Tree



→ Types of Parser

① Universal

- can parse any type/class of grammars, generalized, costly.

② Top down

- parse trees are built from root node to top node
- scan parse trees from left to right
- can be built for LL grammars (LL → class)
↳ left to left derivation

③ Bottom up (rightmost derivation in reverse)

- parse trees are built from bottom to root node
- can be built for LR grammars (LR → class)
↳ left to right derivation

Syntax Error Handling

⇒ Goals of Error Handler

- ① It should report presence of errors clearly & accurately.
- ② It should recover from the errors quickly to detect subsequent errors.

- ③ It should not slow down the process of compilation

start symbol of grammar is always the root node of the grammar

Error Recovery strategies

① Panic Mode

Parser discards input from tokens, synchronizing tokens are formed

Eg:- `int a * /;`


```
int a, b; a;
```

② Phrase level

Pastor performs local corrections on remaining
y/p's

Eg:- `int a+b;`
~~`int a,b;`~~ [replacement]
 instead `a, b;` [efficient]

⑤ Error Productions

Constitutes all errors once productions are feeded to throw most common errors listed to report to us

Eg:- Varde $\rightarrow$ DT idlist;
Var declaration $\rightarrow$ backtracks on reaching(;)

④ Global Corrections

$\Rightarrow$ Context free grammar (CFG)

→ It has set of rules by which valid sentences in a language is constructed. Used for specification for syntactic structure of a programming language.

→ consists of Terminal (basic symbols that can't be replaced) used to form if strings.

→ Token names

→ Non-T (A grammar symbol that can be replaced or expanded to a seq. of symbols) imposes hierarchical structure on the language. They are key to syntax analysis & syntax directed

translation.

→ Start symbol: A Non-Terminal's production is first listed in a grammar

→ Production: specify the grammar rule that describes how to replace or exchange symbols.

⇒ Derivations

→ Always begins w. start symbol

→ leftmost derivation: leftmost production is replaced.

→ Rightmost derivation: rightmost product is replaced.

$$E_x: E \rightarrow E+E \mid E * E \mid -E \mid id \mid \epsilon$$

$$\alpha) -(id + id)$$

$$E \Rightarrow -E$$

$$\Rightarrow -(E)$$

$$\Rightarrow -(E+E)$$

$$\Rightarrow -(id + E)$$

$$\Rightarrow -(id + id)$$

leftmost
derivation

$$E \Rightarrow -E$$

$$\Rightarrow -(E)$$

$$\Rightarrow -(E+E)$$

$$\Rightarrow -(E + id)$$

$$\Rightarrow -(id + id)$$

rightmost
derivation

⇒ Parse Tree

→ Always root node of the parse tree = start symbol

Eg :- $-(id + id)$

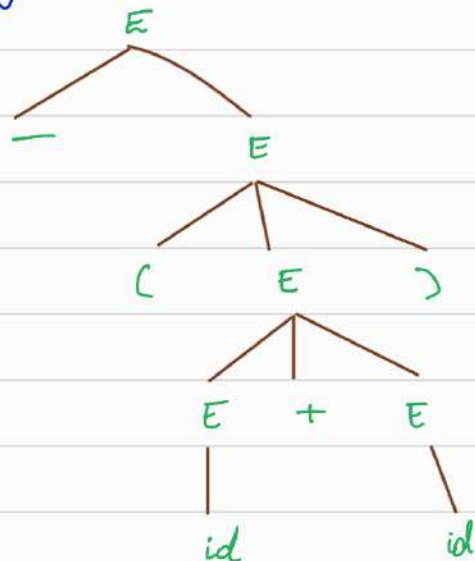
$$E \Rightarrow -E$$

$$\Rightarrow -(E)$$

$$\Rightarrow -(E+E)$$

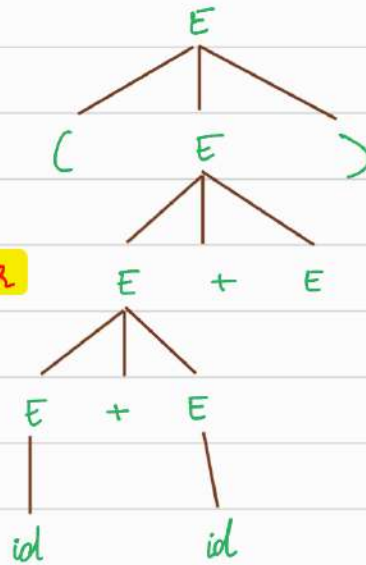
$$\Rightarrow -(id + E)$$

$$\Rightarrow -(id + id)$$



Ex: $(id + id +)$ → throws error as does not consists of a replacement

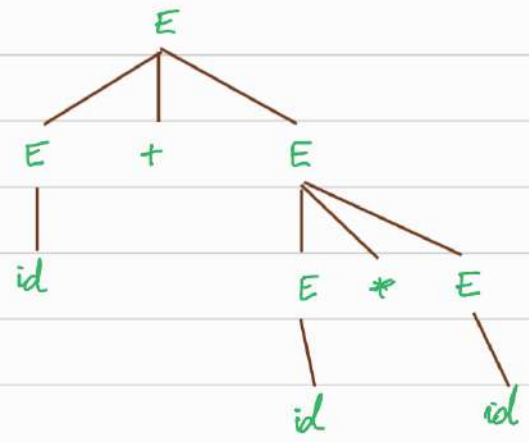
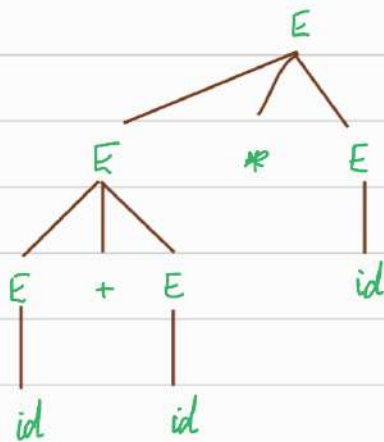
Error occurs when there's an extra character or character does not match



31/1/26

⇒ Ambiguity

Ex: $E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid id$
string :- $id + id * id$



→ Writing a grammar

⇒ Techniques to re-write

- ① Eliminating Ambiguity
- ② Eliminating left Recursion
- ③ Perform left factoring

Now Using Technique ①: Eliminating Ambiguity

NOTE: $\text{start} \rightarrow$
statement

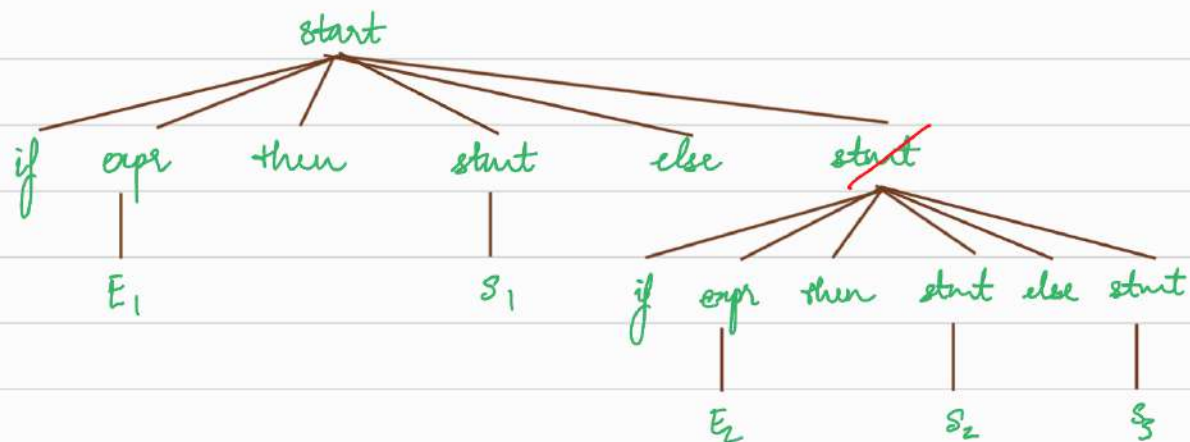
Ex :- Consider G

$\text{start} \rightarrow \text{if expr then start}$

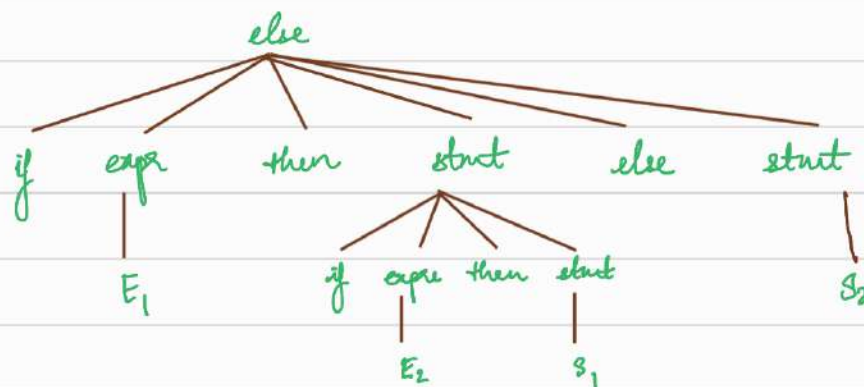
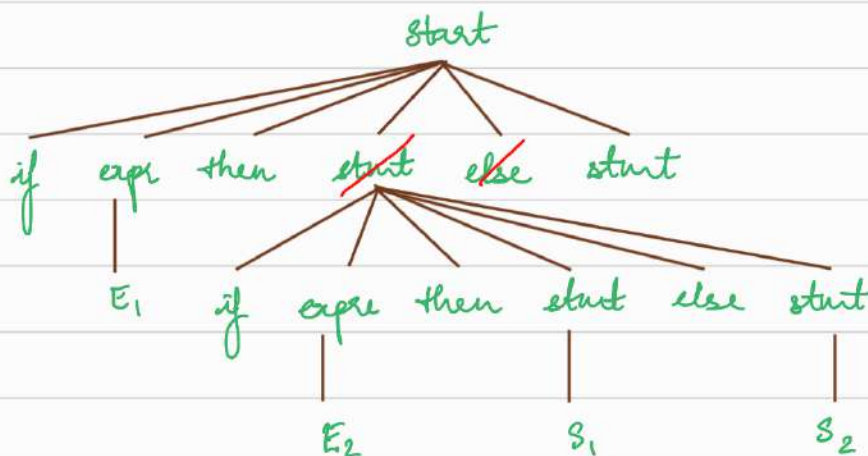
| $\text{if expr then start else start}$ (conditional start)
| other (unconditional start)

Construct Parse tree for

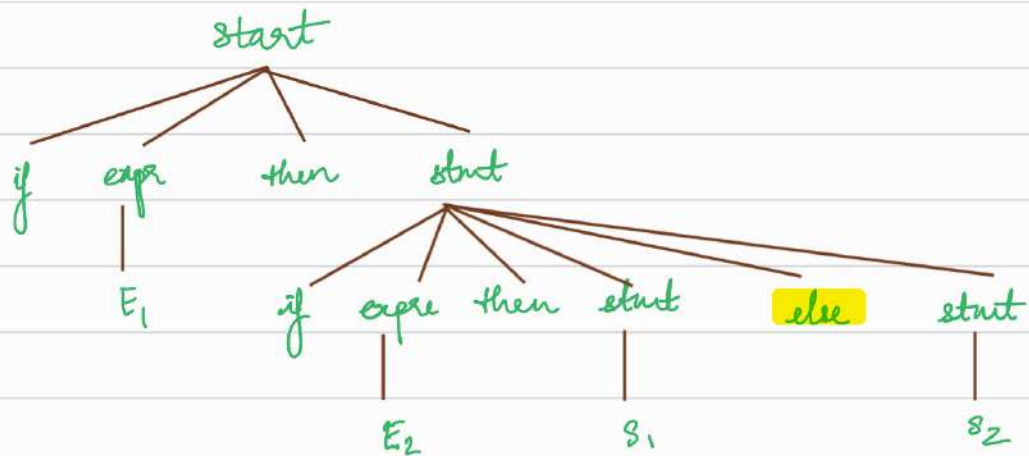
$\text{if } E_1 \text{ then } S_1 \text{ else if } E_2 \text{ then } S_2 \text{ else } S_3$ [Not Ambiguous]



Ex : $\text{if } E_1 \text{ then if } E_2 \text{ then } S_1 \text{ else } S_2$ [ambiguous grammar]

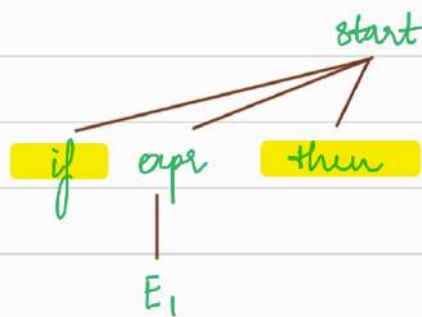


→ Dangling else Problem



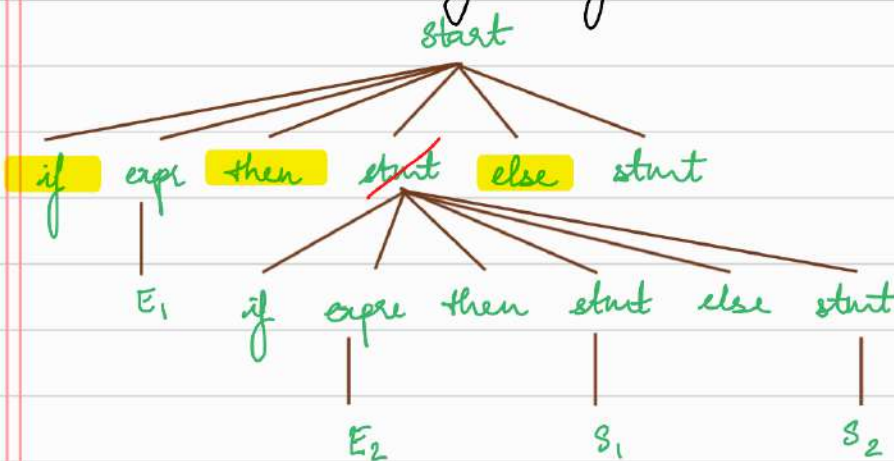
→ Techniques to rewrite grammar

- ① statements b/w then & else is always matched stmt.



matched stmt → if expr then matched stmt
 else matched stmt
 / other

- ② unmatched stmt always has if then else.



unmatched stmt →
 | if expr then stmt
 | if expr then
matched stmt
 else
unmatched stmt

- ③ Only matched stmt is leading to terminal.

NOTE :- language of the grammar should be maintained

② Elimination of left Recursion

A grammar is left recursive if a non-terminal on the left-side appears as first symbol on right-side of the same production.

This type of grammar is not suitable for top-down parsing as it may go for infinite loop as in,

$$A \rightarrow A \alpha / \beta$$

$$A \rightarrow \beta \mid A\alpha \quad (\text{not suitable for top-down})$$

↳ so we re-write

$$A \rightarrow A\alpha \mid \beta$$

⇓

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

Ex: Identify left recursive productions for,

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

A:

$$E \rightarrow E \overset{\alpha_1}{+} \overset{\beta_1}{T} \mid T \quad [\text{It is left recursive, } E \rightarrow E]$$

$$T \rightarrow T \overset{\alpha_2}{*} \overset{\beta_2}{F} \mid F \quad [\text{It is left recursive, } T \rightarrow T]$$

$$F \rightarrow (E) \mid id \quad [\text{Not left recursive}]$$

$$\alpha_1 = +T; \beta_1 = T; \alpha_2 = *F; \beta_2 = F$$

$$E \rightarrow TE'$$

$$T \rightarrow FT'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id \quad [\text{will be retained as it is}]$$

$$A \rightarrow A\alpha \mid \beta$$

$$\Rightarrow A \rightarrow \beta A'$$

$$\Rightarrow A' \rightarrow \alpha A' \mid \epsilon$$

* when β is not given, $\beta = \epsilon$

Ex:

$$S \longrightarrow Aa \mid b$$

$$A \longrightarrow Ac \mid Aad \mid bd \mid l$$

A:

$$\alpha_1 = c ; \alpha_2 = ad ; \beta_1 = bd ; \beta_2 = l$$

$$S \longrightarrow Aa \mid b$$

$$A \longrightarrow bd \mid A' \mid lA'$$

$$A' \longrightarrow cA' \mid adA' \mid \epsilon$$

Ex:

$$S \longrightarrow a \mid \uparrow \mid (T) \mid T$$

$$T \longrightarrow T, S \mid a \mid \uparrow \mid T$$

A:

$$\alpha_1 = ,S ; \beta_1 = a ; \beta_2 = \uparrow ; \beta_3 = (T)$$

$$S \longrightarrow a \mid \uparrow \mid (T) \mid T$$

$$T \longrightarrow aT' \mid \uparrow T' \mid (T)T'$$

$$T' \longrightarrow ,ST' \mid \epsilon$$

~~***~~

$$A \longrightarrow \alpha A \mid \beta$$

$$\Rightarrow A \longrightarrow \beta A'$$

$$\Rightarrow A' \longrightarrow \alpha A' \mid \epsilon$$

Ex:

$$E \longrightarrow \underbrace{E+E}_{\alpha_1} \mid \underbrace{E * E}_{\alpha_2} \mid \underbrace{-E}_{\beta_1} \mid \underbrace{(E)}_{\beta_2} \mid \underbrace{id}_{\beta_3}$$

$$E \longrightarrow -E E' \mid (E) E' \mid id E'$$

$$E' \longrightarrow +E E' \mid * E \mid \epsilon$$

3/2/26

Ex:

$$stmt \longrightarrow \underbrace{\text{if expr then stmt}}_{\alpha_1} \underbrace{\text{else stmt}}_{\beta_1}$$

$$\beta_2 = ? = \epsilon$$

$$A \longrightarrow \alpha \beta_1 \mid \alpha \beta_2$$

$\Downarrow$

$$A \longrightarrow \alpha A'$$

$$A' \longrightarrow \beta_1 \mid \beta_2$$

Only to use for
left factoring

A:

$$stmt \longrightarrow (\text{if expr then stmt})(stmt)'$$

$$(stmt)' \longrightarrow (\text{else stmt}) \mid \epsilon$$

$$[\because A \longrightarrow \alpha A']$$

$$[\because A' \longrightarrow \beta_1 \mid \beta_2]$$

Ex:

$$S \longrightarrow iEtS \mid iEtSeS \mid a$$

$$E \longrightarrow b \text{ (retain)}$$

A:

$$\alpha_1 = iEtS ; \beta_1 = eS ; \beta_2 = \epsilon$$

$$S \longrightarrow iEtS \mid S' \quad \textcircled{a} \rightarrow S \rightarrow a \text{ (retained as it is w. a pipe)}$$

$$S' \longrightarrow eS \mid \epsilon$$

$$E \longrightarrow b$$

Ex:

$$E \longrightarrow E \text{ Sub } E \text{ Sup } E$$

$$E \longrightarrow E \text{ Sub } E$$

$$E \longrightarrow E \text{ Sup } E$$

$$E \longrightarrow \{E\}$$

$$E \longrightarrow e$$

A:

$$E \longrightarrow \underbrace{E E'}_{\alpha_1} \mid \underbrace{\{E\}}_{\text{retain}} \mid \underbrace{e}_{\text{retain}}$$

$$E' \longrightarrow \underbrace{\text{Sub } E \text{ Sup } E}_{\alpha_2} \mid \underbrace{\text{Sub } E}_{\alpha_2} \mid \text{Sup } E$$

$$E' \longrightarrow \underbrace{\text{Sub } E E''}_{\alpha_2} \mid \text{Sup } E$$

simplified by extending prod.

$$E'' \longrightarrow \text{Sup } E \mid \epsilon$$

final solution,

$$E \longrightarrow E E' \mid \{E\} \mid e$$
$$E' \longrightarrow \text{Sub } E E'' \mid \text{Sup } E$$
$$E'' \longrightarrow \text{Sup } E \mid \epsilon$$

Ex:

$$A \longrightarrow \underbrace{abB}_{\beta_1} \mid \underbrace{ab}_{\beta_2} \mid \underbrace{cdB}_{\beta_3} \mid \underbrace{cdeB}_{\beta_4} \mid \underbrace{cdB}_{\beta_5}$$

$$B \longrightarrow b \mid bB$$

A:

$$A \longrightarrow$$

→ Top down

- parse trees are built from root node to top node
- Scan parse trees from left to right
- can be built for LH grammars (LH → class)
↳ left to left derivation

↳ two forms :- ① Backtracking
② Predictive

Both of these use first k follow sets for constructing lookahead

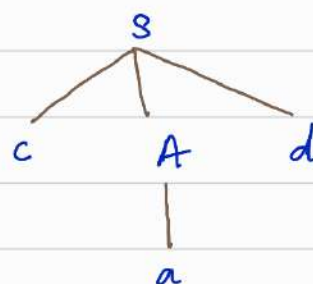
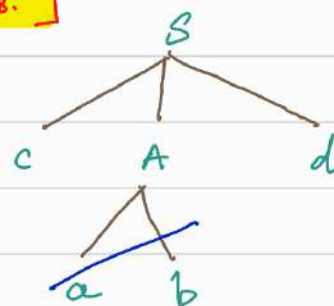
→ Recursive Descent parsers (RDP)
→ Table Driven or LH(1) parsers

→ (Following in lab)

① Backtracking Parsers [powerful & exp. needs lot of time, computationally not poss.]

$S \rightarrow cAd$
 $A \rightarrow ab|a$

input $c a d$
↑ ↗ goes up a level, retracks

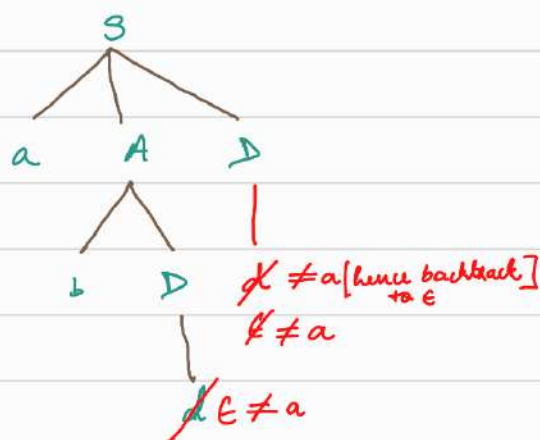


NOTE:- Parse trees are passed pre-order

Ex:

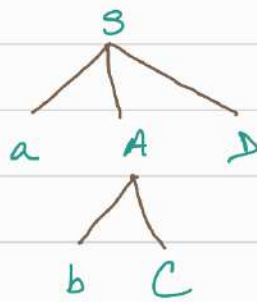
$S \rightarrow aAD | aAC | aCD$
 $A \rightarrow bD | bC | \epsilon$
 $C \rightarrow b | bDa$
 $D \rightarrow d | \epsilon$

input → $ab d a$
 ↑
 matched pointer



[not Successful Parsing]

hence Backtrack to a level up to A



7/2/26

NOTE:- RDP has procedures for all productions, for each non-terminal there is a function (procedure).

$A \rightarrow XaB|b$

$A() \{$

$x()$

Ex: $E \rightarrow \text{num}T$

$T \rightarrow ^*\text{num}T \mid \epsilon$

2 non-terminals = 2 functions

NOTE:- In Top down parsing the '\$' always denotes end of i/p.

Procedure $E \{$

if lookahead = num then {

$E \rightarrow \text{num}T$

(True, calls T)

lookahead = next token;

$T();$

}

else {

error;

}

}

Procedure T {

```

if lookahead = '*' then {
    if lookahead = num then {
        lookahead = next token;
        T();
    }

```

```

    else {
        error;
    }
}

```

else NUL;

}

Procedure match (token) {

```

    if lookahead = token {
        lookahead = next token;
    }
    else {
        error;
    }
}

```

}

Procedure error {

```

    printf ("error");
}

```

Main() {

E();

```

    if lookahead = $ {

```

Declare success;

}

```

    else {

```

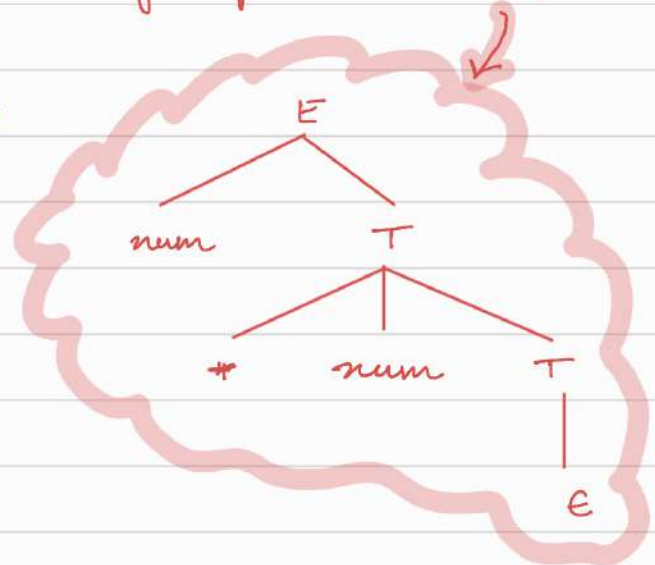
error;

}

}

$T = * num T$ (true, calls T)

now for input: num * num \$



Backlook algorithm for
Procedure T

Predictive Parser

→ uses first & follow

→ consider 2 prod.'s $A \rightarrow \alpha | \beta$

If current i/p symbol is 'a' we choose prod. ' α ', if 'a' is present in $\text{first}(\alpha)$.

→ $\text{FIRST}(\alpha)$: set of terminals that can be beginning of strings, derived from α .

$\text{FOLLOW}(A)$: set of terminals that occur to right of A.

Rules for calculation for FIRST

1. If ' α ' is a terminal 'a' then $\text{FIRST}(\alpha) = \{a\}$
2. If ' α ' is a non-terminal ' X ' and $\alpha \rightarrow a\beta$ then $\text{FIRST}(\alpha) = \{a\}$
3. If ' α ' is a non-terminal ' X ' and $\alpha \rightarrow \epsilon$ then $\text{FIRST}(\alpha) = \{\epsilon\}$
4. If $\alpha \rightarrow y_1 y_2 y_3 \dots y_k$ then place $\text{FIRST}(y_1)$ in $\text{FIRST}(\alpha)$:
 - ① If $\text{FIRST}(y_1)$ has ϵ then add $\text{FIRST}(y_2)$ and so on
 - ② If ϵ is in $\text{FIRST}(y_j)$ where $j = 1, 2, \dots, k$ then add ϵ to $\text{FIRST}(\alpha)$.

Ex:

$X \rightarrow y_1 y_2 y_3$

$y_1 \rightarrow a | \epsilon$

$y_2 \rightarrow b | \epsilon$

$y_3 \rightarrow c | \epsilon$

Ans:

	first	follow
X	$\{a, b, c, \epsilon\}$	
y_1	$\{a, \epsilon\}$	
y_2	$\{b, \epsilon\}$	
y_3	$\{c, \epsilon\}$	

$X \rightarrow y_1 y_2 y_3 \epsilon$
 $y_1 \rightarrow a | \epsilon$
 $y_2 \rightarrow b | \epsilon$
 $y_3 \rightarrow c | \epsilon$

10/2/26

Ex:

$x \rightarrow y_1 y_2 y_3$

$y_1 \rightarrow a | \epsilon$

$y_2 \rightarrow b | \epsilon$

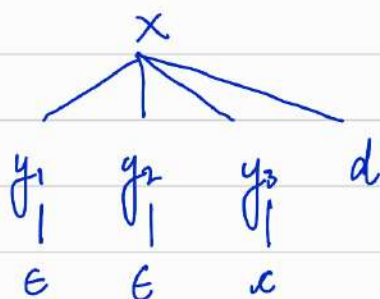
$y_3 \rightarrow c | \epsilon$

A:

	first	follow
x	a, b, c, d	
y ₁	a, ϵ	
y ₂	b, ϵ	
y ₃	c, ϵ	

$x \rightarrow \cancel{y_1} \cancel{y_2} \cancel{y_3} d$
 $y_1 \rightarrow a | \epsilon$
 $y_2 \rightarrow b | \epsilon$
 $y_3 \rightarrow c | \epsilon$

i/p - c



Rules for FOLLOW SET

- Whenever you want to follow of a symbol, find the symbol in R.H.S.
- If symbol is start symbol, add $\$$ to follow set.
- If symbol is followed by a terminal, straightway add the terminal to the follow set.
- If the symbol is followed by non-terminal, find the first of the non-terminal
 - If the first set contains ϵ , exclude ϵ from follow sets & replace it in the production.
 - If even after replacement by ϵ , it is followed by terminal/non-terminal, repeat the process.
- If the start symbol is last one in the prod., then add follow of head into follow set.

NOTE :- FOLLOW set do not contain ' ϵ '

Ex d:

	first	follow
X	a, b, c, d	{ \$ }
y ₁	a, ε	b, c, d
y ₂	b, ε	c, d
y ₃	c, ε	d

sol

$x \rightarrow y_1 y_2 y_3 d$
 $y_1 \rightarrow a | \epsilon$
 $y_2 \rightarrow b | \epsilon$
 $y_3 \rightarrow c | \epsilon$

trailing

i/p - c | ε

Ex 1:

	first	follow
X	{ a, b, c, ε }	\$
y ₁	{ a, ε }	b, c, \$
y ₂	{ b, ε }	c, \$
y ₃	{ c, ε }	\$

sol

$x \rightarrow y_1 y_2 y_3 \epsilon$
 $y_1 \rightarrow a | \epsilon$
 $y_2 \rightarrow b | \epsilon$
 $y_3 \rightarrow c | \epsilon$

trailing

i/p - c | ε

Ex 3:

Q) $\begin{cases} s \rightarrow uw \\ u \rightarrow a | \epsilon \\ w \rightarrow d | \epsilon \end{cases}$

	first	follow
S	{ a, d, ε }	\$
u	{ a, ε }	d, \$
w	{ d, ε }	\$

sol

Ex 4:

$E \rightarrow E + T | F$
 $T \rightarrow T * F | F$
 $F \rightarrow (E) | id$

$E \rightarrow E + T | \underline{T}$ [It is left recursive, $E \rightarrow E$]
 $T \rightarrow T * F | \underline{F}$ [It is left recursive, $T \rightarrow T$]
 $F \rightarrow (E) | id$ [Not left recursive]

$K_1 = +T; P_1 = T; K_2 = *F; P_2 = F$

$E \rightarrow TE'$

$T \rightarrow FT'$

$E' \rightarrow +TE' | \epsilon$

$T' \rightarrow *FT' | \epsilon$

$F \rightarrow (E) | id$ [will be retained as it is]

* when β is not given, β = ε

~~***~~
 $A \rightarrow AK | \beta$
 $\Rightarrow A \rightarrow \beta A'$
 $\Rightarrow A' \rightarrow K A' | \epsilon$

	first	follow
E	{(, id}	{\$, }
E'	{+, ε}	{\$, }
T	{(, id}	{+, }, \$}
T'	{*, ε}	{+, }, \$}
F	{(, id}	{+, *, (,), \$}

$$E \rightarrow \underline{T} E'$$

$$T \rightarrow \underline{F} T'$$

$$E' \rightarrow \underline{+} T E' \mid \underline{\epsilon}$$

$$T' \rightarrow \underline{*} F T' \mid \underline{\epsilon}$$

$$F \rightarrow \underline{(} E \mid \underline{id}$$

1.

follow

$\$$
 $T, \$$
 $\$$

first
 $\{a, T, \epsilon, \epsilon\}$

S
 T
 T'
 $\{S, T', \epsilon\}$

$S \rightarrow a | \epsilon | (T)$
 $P_1 \quad P_2 \quad P_3$

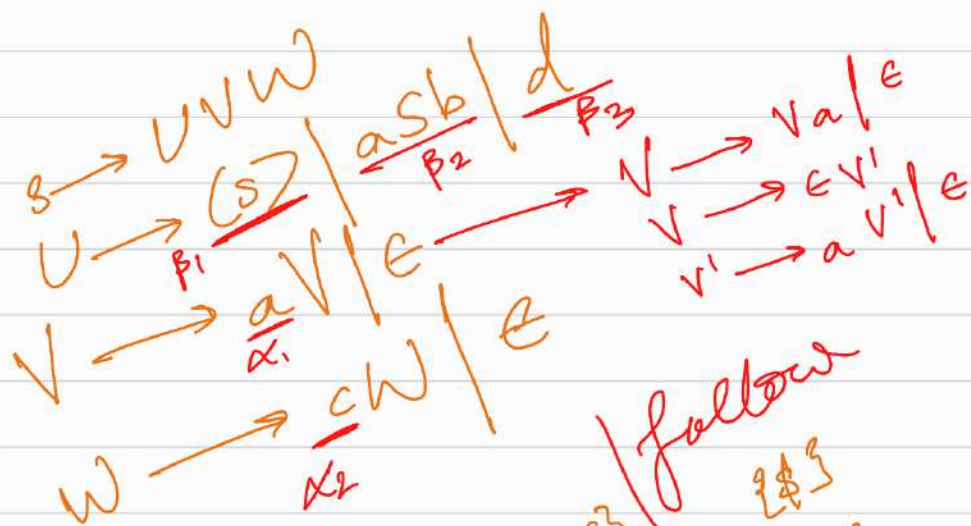
$S \rightarrow a | \epsilon | (T)$
 $T \xrightarrow{\alpha_1} T S | S$
 β ~~action~~

$T \rightarrow T \alpha_1 | P_4$

$T \rightarrow T S | \epsilon$

$T \rightarrow \epsilon T'$
 $T' \rightarrow S T' | \epsilon$

2.



first

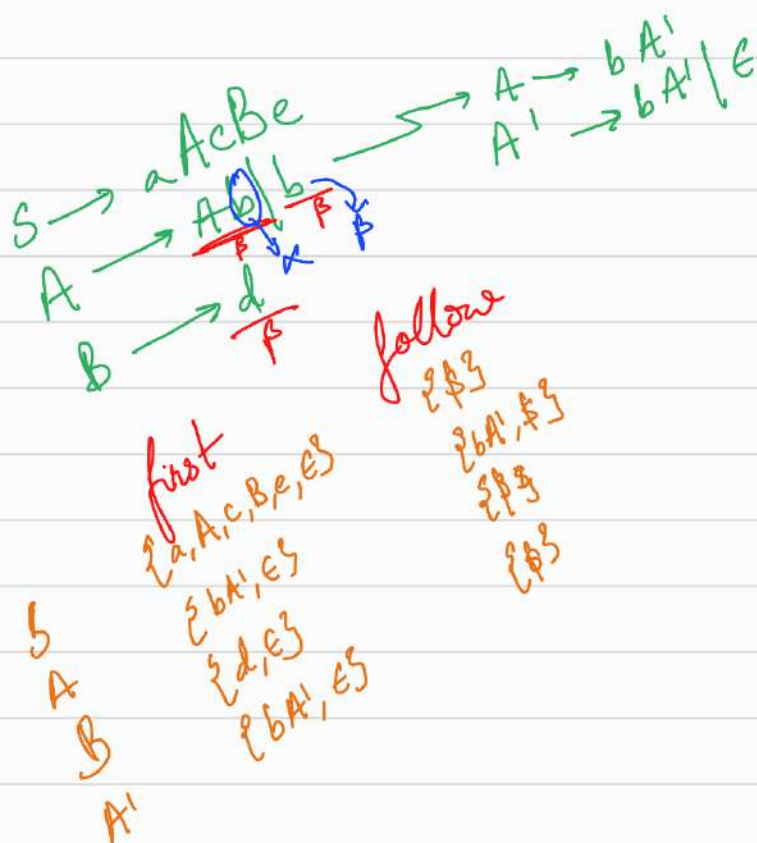
$\{ \epsilon, ev', ew', av', aw', \epsilon \}$

S
 U
 V
 W
 V'
 W'

follow

$\{ \epsilon \}$
 $\{ \epsilon \}$
 $\{ v', \epsilon \}$
 $\{ w', \epsilon \}$
 $\{ \epsilon \}$
 $\{ \epsilon \}$

3.



4.

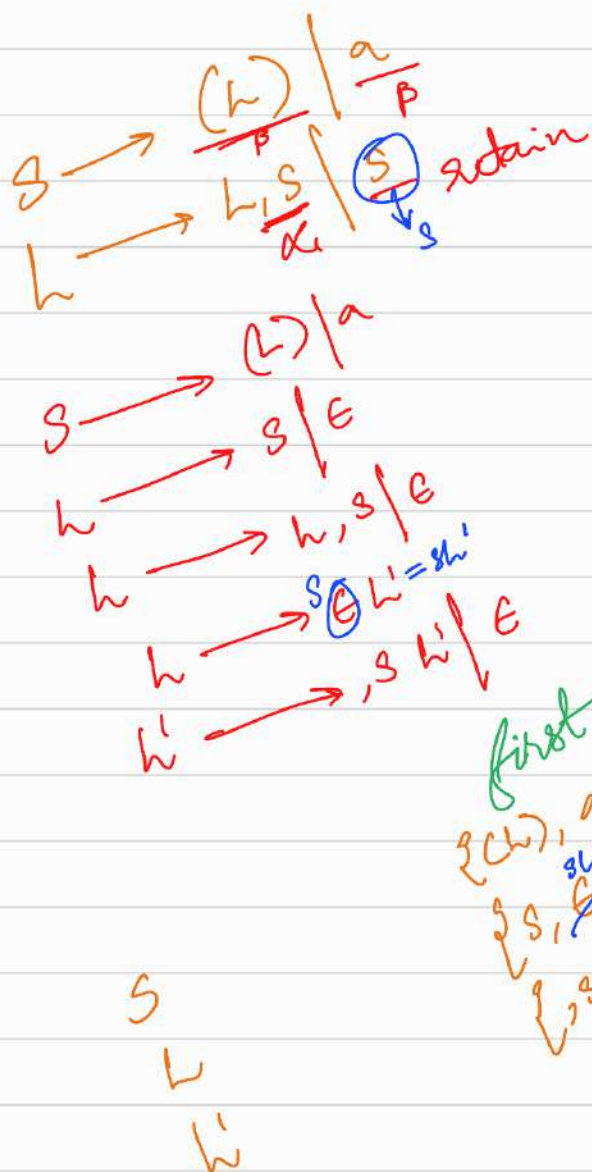
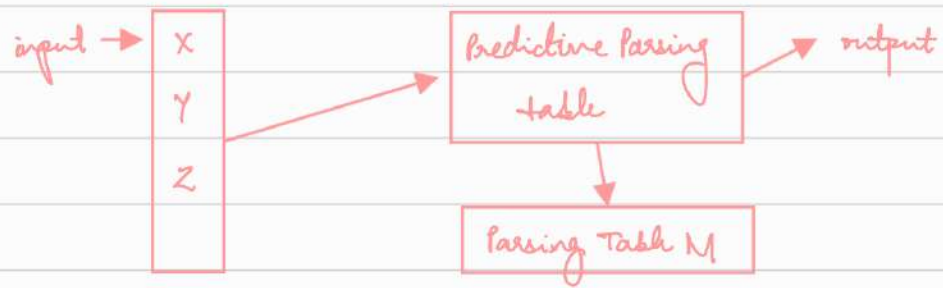


Table Driven Predictive Parser



02/12/20

Ex:

$A \rightarrow BC$

$B \rightarrow S$

$C \rightarrow ; A | \epsilon$

A:

	first	follow
A	S	\$
B	S	;\$
C	;,ε	\$

M[i,j]	S	;	\$
A	$A \rightarrow BC$		
B	$B \rightarrow S$		
C		$C \rightarrow ;A$	$C \rightarrow \epsilon$

NOTE:-

— : pop

↑ : pointer

Stack	Input	Action
A\$	ε ; S\$	
BC\$	S ; S\$	$A \rightarrow BC$
SC\$	S ; S\$	$B \rightarrow S$
C\$	; S\$	Match S
;A\$	; S\$	$C \rightarrow ;A$
A\$	S\$	Match ;
BC\$	S\$	$A \rightarrow BC$
SC\$	S\$	$B \rightarrow S$
C\$	\$	Match S
\$	\$	$C \rightarrow \epsilon$ Success

Ex:

$S \rightarrow AaAb \mid BbBa$
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$

input string: ab

A:

The grammar is not unambiguous

— $\Rightarrow$ — is not left recursive

— $\Rightarrow$ — is not needed for left-factoring

	first	follow
S	a, b	\$
A	ϵ	a, b
B	ϵ	a, b

M[i]	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	

* if there is ϵ in first, make entry in follow set

Stack	Input	Action
S\$	ab\$	
<u>A</u> aAb\$	ab\$	$S \rightarrow AaAb$
aAb\$	ab\$	$A \rightarrow \epsilon$
Ab\$	ab\$	Match a
b\$	b\$	$A \rightarrow \epsilon$
\$	b\$	Match b
	\$	Success

Ex:

$$\begin{aligned} E &\longrightarrow E+T \mid T \\ T &\longrightarrow T*F \mid F \\ F &\longrightarrow (E) \mid id \end{aligned}$$

i/p :- id + id * id

A:

grammar is not ambiguous
grammar is left recursive

$$\begin{aligned} E &\longrightarrow E+T \mid T \quad [It is left recursive, E \rightarrow E] \\ T &\longrightarrow T*F \mid F \quad [It is left recursive, T \rightarrow T] \\ F &\longrightarrow (E) \mid id \quad [Not left recursive] \end{aligned}$$

$$K_1 = +T, P_1 = T, K_2 = *F, P_2 = F$$

$$E \longrightarrow TE'$$

$$T \longrightarrow FT'$$

$$E' \longrightarrow +TE' \mid E$$

$$T' \longrightarrow *FT' \mid E$$

$$F \longrightarrow (E) \mid id \quad [will be retained as it is]$$

* when β is not given, $\beta = \epsilon$

$$\begin{aligned} A &\longrightarrow A\alpha \mid \beta \\ \Rightarrow A &\longrightarrow \beta A' \\ \Rightarrow A' &\longrightarrow \alpha A' \mid \epsilon \end{aligned}$$

grammar doesn't need left factoring

occurrence of E followed again in F followed by ')

deriving follow from parent terminals

	first	follow
E	{(, id}	{\$,)}
E'	{+, ε}	{\$,)}
T	{(, id}	{+,), \$, ε}
T'	{*, ε}	{+,), \$, T}
F	{(, id}	{+, *, (,), \$}

$$E \longrightarrow TE' \mid ? \rightarrow \text{follow } E$$

$$T \longrightarrow FT'$$

$$E' \longrightarrow +TE' \mid E$$

$$T' \longrightarrow *FT' \mid E$$

$$F \longrightarrow (E) \mid id$$

$$T \longrightarrow E' \longrightarrow E$$

follows

Parse :-	M[]	(	+	*	)	\$	id
Table	E	E → TE'					E → TE'
	E'		E' → +TE'		E' → ε	E' → E	
	T	T → FT'					T → FT'
	T'			T' → *FT'	T' → ε	T' → E	
	F	F → (E)					F → id

Stack	Input	Action
$\underline{E} \$$	$id + id * id \$$	
$\underline{T}E' \$$	$id + id * id \$$	$E \rightarrow TE'$
$\underline{F}T'E' \$$	$id + id * id \$$	$T \rightarrow FT'$
$\underline{id}T'E' \$$	$id + id * id \$$	$F \rightarrow id$
$\underline{T'E' \$}$	$id + id * id \$$	Match id
$\underline{E' \$}$	$+ id * id \$$	$T' \rightarrow E$
$\underline{+TE' \$}$	$+ id * id \$$	$E' \rightarrow +TE'$
$\underline{T}E' \$$	$+ id * id \$$	Match +
$\underline{F}T'E' \$$	$id * id \$$	$T \rightarrow FT'$
$\underline{id}T'E' \$$	$id * id \$$	$F \rightarrow id$
$\underline{T'E' \$}$	$id * id \$$	Match id
$\underline{*FT' \$}$	$* id \$$	$T' \rightarrow *FT'$
$\underline{F}T' \$$	$* id \$$	Match *
$\underline{id}T' \$$	$id \$$	$F \rightarrow id$
$\underline{T' \$}$	$id \$$	Match id
$\underline{\$}$	$\$$	$T' \rightarrow \leftarrow$ Success

popping T,
pushing E



As there are no

2)

Program $\rightarrow$ main ($\rightarrow$) { declarations assign-stat }

declarations $\rightarrow$ data-type identifier-list; declarations | ϵ

data-type $\rightarrow$ int | char

identifier-list $\rightarrow$ id | id, identifier-list

assign-stat $\rightarrow$ id = id; | id = num

declarations $\rightarrow$ data-type identifier-list; declarations | ϵ

κ = data-type identifier-list;

~~★★~~

$A \rightarrow A\kappa | B$

$\Rightarrow A \rightarrow \beta A'$

$\Rightarrow A' \rightarrow \kappa A' | \epsilon$

declarations $\rightarrow \epsilon$ declarations'

declarations' $\rightarrow$ data-type identifier-list; declarations' | ϵ

Program $\rightarrow$ ^Tmain ($\rightarrow$) { ^{NT}declarations ^Tassign-stat }

declarations $\rightarrow \epsilon$ declarations'

declarations' $\rightarrow$ data-type identifier-list; declarations' | ϵ

data-type $\rightarrow$ int | char

identifier-list $\rightarrow$ id identifier-list'

identifier-list' $\rightarrow$ id, identifier-list' | ϵ

assign-stat $\rightarrow$ id = id; | id = num

if (lookahead == first(declarations)) {
 if (@ as in ϵ) {
 else if (first(ϵ) follows (declarations)) {
 return; }
}

3/2/26

Ex:

Check whether given grammar is LL(1).

$$S \rightarrow UVW$$

$$U \rightarrow (s) \mid asb \mid d \mid e$$

$$V \rightarrow aV \mid e$$

$$W \rightarrow cW \mid e$$

Ans:

given grammar is not Ambiguous

given grammar is not left recursive

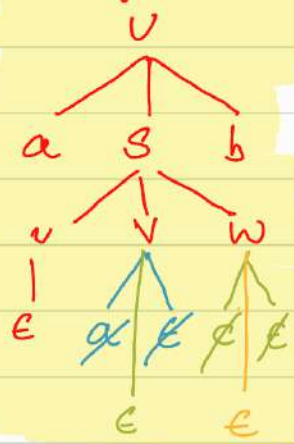
given grammar need not undergo left factoring

First(S) $\rightarrow$ First(U)
First(U) $\rightarrow$ consists of ϵ
Hence, First(S) $\rightarrow \epsilon$

	first	follows
S	(, a, d, c, e	\$, >, b
U	(, a, d, e	a, c, \$, >, b
V	a, e	c, >, b, \$
W	c, e	\$, >, b

M[]	(	a	d	c	\$	>	b
S	$S \rightarrow UVW$	$S \rightarrow UVW$	$S \rightarrow UVW$	$S \rightarrow UVW$	$S \rightarrow UVW$	$S \rightarrow UVW$	$S \rightarrow UVW$
U	$U \rightarrow (s)$	$U \rightarrow asb$ $U \rightarrow e$	$U \rightarrow d$	$U \rightarrow e$	$U \rightarrow e$	$U \rightarrow e$	$U \rightarrow e$
V		$V \rightarrow aV$		$V \rightarrow e$	$V \rightarrow e$	$V \rightarrow e$	$V \rightarrow e$
W				$W \rightarrow cW$	$W \rightarrow e$	$W \rightarrow e$	$W \rightarrow e$

First Explanation



NOTE:- Even if grammar is not LL(1), it can be parsed

Since $M[U, a]$ has multiple entries given grammar is not LL(1).

U can be expanded either in



(or)



To compute if grammar is $LL(1)$ or not:

- (i) a) first write FIRST & FOLLOW of the grammar given
- b) check whether you have disjoint follow set
- c) if no common set $\rightarrow$ not $LL(1)$
else $\rightarrow$ $LL(1)$

(ii) $A \rightarrow \alpha / \beta$,

FIRST(α) and FIRST(β) should be disjoint.

if $\beta \xRightarrow{*} \epsilon$,

then FIRST & FOLLOW

sets should be disjoint.

// means β giving ϵ in 'n' no. of steps

Q) $S \rightarrow a \mid \uparrow \mid (T)$

$T \rightarrow T_1 S \mid \frac{S}{\beta}$

Parse: $(a, a) \$$

$A \rightarrow \alpha \mid \beta$

$\Rightarrow A \rightarrow \beta A'$

$\Rightarrow A' \rightarrow \alpha A' \mid \epsilon$

A:

$S \rightarrow a \mid \uparrow \mid (T)$

$T \rightarrow S T'$

$T' \rightarrow , S T' \mid \epsilon$

	first	follow
S	a, $\uparrow$, (	$\$, ,$
T	a, $\uparrow$, (, ,	,, $\$, ,$
T'	,, ϵ	,, $\$$

Ex:

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

} if $p := ba\$$

A:

The grammar is not an ambiguous

_____ is not left recursive

_____ is not needed for left-factoring

	first	follow
S	a, b	\$
A	ϵ	a, b
B	ϵ	a, b

$M[i, j]$	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	

stack	Input	Action
S \$	b a \$	
<u>B</u> b B a \$	b a \$	$S \rightarrow BbBa$
<u>b</u> B a \$	<u>b</u> a \$	$B \rightarrow \epsilon$
<u>B</u> a \$	<u>a</u> \$	Match b
<u>a</u> \$	<u>a</u> \$	$B \rightarrow \epsilon$
\$	\$	Match a Success

Ex:

$$S \rightarrow zMNz$$

$$M \rightarrow \underset{\kappa_1}{a} \underset{\kappa_2}{M} \underset{\beta_1}{a|z}$$

$$N \rightarrow \underset{\kappa_3}{b} \underset{\kappa_4}{N} \underset{\beta_2}{b|z}$$

A:

grammar is not ambiguous

grammar is not left recursive

grammar requires left-factoring

$$S \rightarrow zMNz$$

$$M \rightarrow zM' | zM''$$

$$M' \rightarrow aM' | \epsilon$$

$$M'' \rightarrow aM'' | \epsilon$$

$$N \rightarrow zN' | zN''$$

$$N' \rightarrow bN' | \epsilon$$

$$N'' \rightarrow bN'' | \epsilon$$

	first	follows
S	z	\$
M	z	z, \$
M'	a, ε	a, z, \$
M''	a, ε	a, z, \$
N	z	a, z, \$
N'	b, ε	b, a, z, \$
N''	b, ε	b, a, z, \$

Ex:

$$\begin{aligned} E &\rightarrow EE^+ \mid \beta_1 = \epsilon \\ E &\rightarrow EE^* \mid \beta_2 = \epsilon \\ E &\rightarrow \text{num} \end{aligned}$$

~~***~~

$$\begin{aligned} A &\rightarrow A\alpha \mid \beta \\ \Rightarrow A &\rightarrow \beta A' \\ \Rightarrow A' &\rightarrow \alpha A' \mid \epsilon \end{aligned}$$

A:

grammar is not ambiguous
grammar is not left recursive
grammar requires left-factoring

$$\begin{aligned} E &\rightarrow EE^+ \\ E^+ &\rightarrow E^+ E^+ \mid \epsilon \\ E &\rightarrow EE'' \\ E'' &\rightarrow E^* E'' \mid \epsilon \\ E &\rightarrow \text{num} \end{aligned} \quad \left. \begin{aligned} E &\rightarrow EE^+ \mid EE'' \mid \text{num} \\ E^+ &\rightarrow E^+ E^+ \mid \epsilon \\ E'' &\rightarrow E^* E'' \mid \epsilon \end{aligned} \right\}$$

	first	follow
E	num, ϵ	$\$, +, *$
E ⁺	num, ϵ	$\$, +, *$
E''	num, ϵ	$\$, +, *$

Ex:

$$\begin{aligned} S &\rightarrow iCtSS|a \\ S' &\rightarrow aS \mid \epsilon \\ C &\rightarrow b \end{aligned}$$

A:

Ex:

$$A \longrightarrow S a Y$$

$$S \longrightarrow b | x Y$$

$$X \longrightarrow c | \epsilon$$

$$Y \longrightarrow d | \epsilon$$

Ex:

show compilation phases for follow state

$$X = -(a[i] + b[i] + (\epsilon/y))$$

Ex:

$$r1 = a(a|b)^* a$$

$$r2 = (a|b)^* a (a|b) (a|b)$$

$$r3 = (aa|bb)^* (ab|ba)$$

i/p :- aabbbaba

2 Draw NFA to then
pattern matching

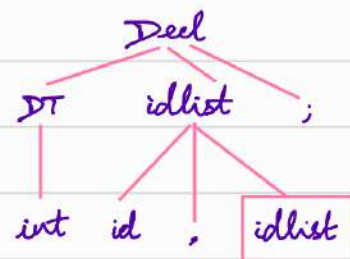
14/3/26

Error Recovery in predictive parsing

Q) How to select synchronizing set?

Step 1: Place all symbols in FOLLOW(A) into the synchronizing set for non-terminal A. If we skip tokens until an element of FOLLOW(A) is seen and pop A.

Eg:-
 $Decl \rightarrow DT \text{ idlist};$
 $\text{idlist} \rightarrow \text{id}, \text{idlist} | \epsilon$
 $DT \rightarrow \text{int} | \text{char}$



	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	synch	synch
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T						
T'						
F						

→ Bottom Up Parsing

→ LR grammars

→ h: left to right

→ R: Rightmost derivation

* not suitable for LR grammars

→ Shift-Reduce Parsers

Underline →

Non-underline →

Eg:- e

Eg:- e

• Grammar:-

$$\underline{e} \longrightarrow (\underline{e} + \underline{e})$$

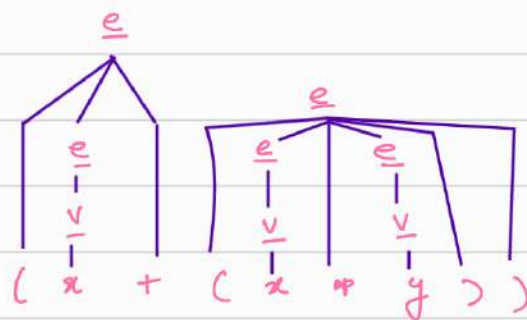
$$\underline{e} \longrightarrow (\underline{e} * \underline{e})$$

$$\underline{e} \longrightarrow v$$

$$\underline{v} \longrightarrow x$$

$$\underline{v} \longrightarrow y$$

• Parse Tree:-



• Input:-

$$(x + (x * y))$$

Ex:-

$$E \longrightarrow E + T \mid T$$

$$T \longrightarrow T * F \mid F$$

$$F \longrightarrow id \mid (E)$$

input:- $id * (id + id)$

A:-

$$E \longrightarrow E + \overset{\alpha_1}{T} \mid \overset{\beta_1}{T} \quad [\text{It is left recursive, } E \rightarrow E]$$

$$T \longrightarrow T * \overset{\alpha_2}{F} \mid \overset{\beta_2}{F} \quad [\text{It is left recursive, } T \rightarrow T]$$

$$F \longrightarrow (E) \mid id \quad [\text{Not left recursive}]$$

$$\alpha_1 = +T; \beta_1 = T; \alpha_2 = *F; \beta_2 = F$$

$$E \longrightarrow TE'$$

$$T \longrightarrow FT'$$

$$E' \longrightarrow +TE' \mid \epsilon$$

$$T' \longrightarrow *FT' \mid \epsilon$$

$$F \longrightarrow (E) \mid id \quad [\text{will be retained as it is}]$$

* when β is not given, $\beta = \epsilon$

$$A \longrightarrow A\alpha \mid \beta$$

$$\Rightarrow A \longrightarrow \beta A'$$

$$\Rightarrow A' \longrightarrow \alpha A' \mid \epsilon$$

$$\therefore E \longrightarrow TE'$$

$$E \longrightarrow TE'$$

$$E' \longrightarrow +TE' \mid \epsilon$$

$$E' \longrightarrow +TE'$$

$$T \longrightarrow FT'$$

$$E' \longrightarrow \epsilon$$

$$T \longrightarrow *FT' \mid \epsilon$$

$$T \longrightarrow FT'$$

$$T' \longrightarrow *FT'$$

Handling Parsing

→ Handle is the substring which matches right side of the prod. & we can reduce such string by a non-terminal on LHS of the production.

→ Reduction of a string or handle by a suitable Non-terminal is called parsing.

* It can have both terminal & Non-terminal

Parser performs following basic operations :-

① Shift (Actions performed by bottom-up parser)

• Moving of the symbols from i/p onto the stack.

consists both terminals & Non-terminals

② Reduce

• If handle appears on top of stack, then reduction of handle it by LHS of the grammar rule/production

③ Accept

• when stack contains stack ^{only} start symbols & i/p buffer is empty.

④ Error

• A situation where either shift reduce nor accept.

~ If using general shift-reduce parser, we have to prioritize shift over reduce in the stack.

Conflicts

① Shift Reduce Conflict

② Reduce Reduce Conflict

① Shift Reduce Conflict

Eg:- Dangling else problem

② Reduce Reduce Conflict

Eg:-
parameter → id
expr → id
stack

.... id(id

solution? ⇒ change it to preced

①

Shift Reduce Parser

→ SLR : simple LR (basic) ★

→ Canonical LR parser ★

→ LALR : lookahead LR parser

→ Items and LR(0) Automation

Ex: $S' \rightarrow S$ $S \rightarrow (S) S \mid \epsilon$ A:

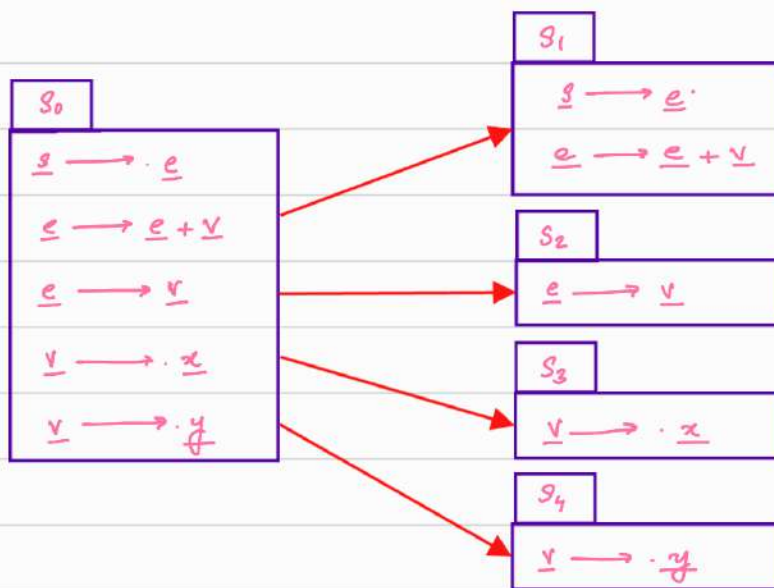
→ Closure of item sets

 $I \rightarrow$ set of items of G closure(I) $\rightarrow$ 2 rulesInitially add every item in I to closure(I)If $A \rightarrow \alpha \cdot B \beta$ is in closure(I) and $B \rightarrow \gamma$ is a

Q

 $\underline{s} \rightarrow \underline{e}$ $\underline{e} \rightarrow \underline{e} + \underline{v}$ $\mid \underline{v}$ $\underline{v} \rightarrow \underline{x}$ $\mid \underline{y}$ what is closure($(\underline{s} \rightarrow \underline{e})$)?* We are using SLR(0)
SLR(0) $\rightarrow$ LR(0) itemsA:= { $\underline{s} \rightarrow \cdot \underline{e}$ } First iteration, $\underline{e} \rightarrow \underline{e} + \underline{v}$ } second iteration, $\underline{e} \rightarrow \underline{v}$ } Third iteration, $\underline{v} \rightarrow \cdot \underline{x}$, $\underline{v} \rightarrow \underline{y}$

}



└

2)

Program $\rightarrow$ main ($\rangle$ { declarations assign-stat }

declarations $\rightarrow$ data-type identifier-list; declarations | ϵ

data-type $\rightarrow$ int | char

identifier-list $\rightarrow$ id | id, identifier-list

assign-stat $\rightarrow$ id = id; | id = num

declarations $\rightarrow$ data-type identifier-list; declarations | ϵ

κ = data-type identifier-list;

~~NT~~

$A \rightarrow A\kappa | B$

$\Rightarrow A \rightarrow \beta A'$

$\Rightarrow A' \rightarrow \kappa A' | \epsilon$

declarations $\rightarrow \epsilon$ declarations'

declarations' $\rightarrow$ data-type identifier-list; declarations' | ϵ

Program $\rightarrow$ main ($\rangle$ { declarations assign-stat } ^T _T

declarations $\rightarrow \epsilon$ declarations'

declarations' $\rightarrow$ data-type identifier-list; declarations' | ϵ

data-type $\rightarrow$ int | char

identifier-list $\rightarrow$ id identifier-list'

identifier-list' $\rightarrow$ id, identifier-list' | ϵ

assign-stat $\rightarrow$ id = id; | id = num

$\text{if (lookahead} == \text{first(declarations)) \{$
 $\text{if (@ as in } \epsilon) \{$
 $\text{else if (first } \epsilon) \text{ follows (declarations)) \{$
 $\text{return; \}}$

expr $\rightarrow$ simple-expr eprime

epime $\rightarrow$ relop simple-expr | ϵ

simple-op $\rightarrow$ term seprime

seprime $\rightarrow$ addop term seprime | $\frac{\epsilon}{\beta} \Rightarrow$ seprime $\rightarrow \epsilon$ seprime'

seprime' $\rightarrow$ addop term seprime' | ϵ

term $\rightarrow$ factor tprime

tprime $\rightarrow$ mulop factor tprime | ϵ

factor $\rightarrow$ id | num

decision-stat $\rightarrow$ if (expr) {statement-list} | ϵ

relop $\longrightarrow$ $==$ $!=$ $<=$ $>=$ $>$ $<$

addop $\longrightarrow$ $+$ $-$

mulop $\longrightarrow$ $*$ $/$ $\%$

statement $\longrightarrow$ assign-stat; | decision-stat | looping-stat

decision-stat $\longrightarrow$ if (expn) {statement-list} dprime

* lab eval 2 : 26th March (3:30 - 4:30 pm, lab 1-8 (except lab 7 Run))

lab obs.

6/20

lab midsem

3/28

[evaluation
makes
pending]

niva, (4+4+4)

/12 $\Rightarrow$ /60

external (15+25) $\Rightarrow$ /40

$\Rightarrow$ /100

19/3/26
→

SLR(1)

Q) $E \rightarrow E+n/n$

A:

Step 1: Augment the grammar

$E' \rightarrow E$

$E \rightarrow E+n$

$E \rightarrow n$

Step 2: No. from zero

0) $E' \rightarrow E$

1) $E \rightarrow E+n$

2) $E \rightarrow n$

Step 3: Build LR(0) Automata (DFA)

Step 4: Finding the start state of DFA

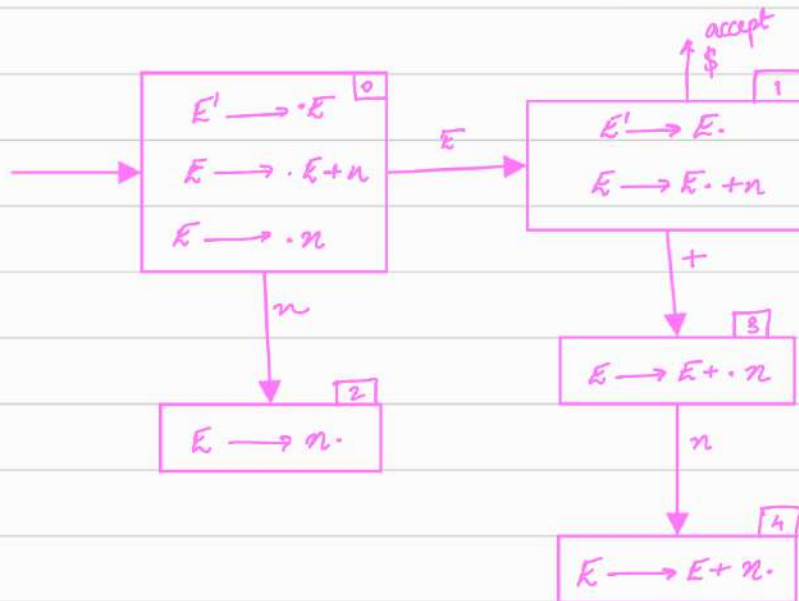
obtained by,

closure ($E' \rightarrow \cdot E$)

$\{E' \rightarrow \cdot E\}$ non-terminal

$E \rightarrow \cdot E+n$

$E \rightarrow \cdot n$



NOTE:- Row headers are states no's. & col headers are symbols (terminals & nonterminals)

SLR(1) Parse Table

	Action			Goto	
	n	$+$	$\$$	E'	E
0	S_2				1
1		S_3	accept		
2		$r2$	$r2$		
3	S_4				
4		$r1$	$r1$		

★ S_2 for instance, here $S \rightarrow$ shift & no. associated $\rightarrow$ state no.
 whereas reduce $\rightarrow$ prod. rule no.

★ Start again for stack is $\$$ & start state

pop 3 sym. from stack

Stack	symbols	Input	Action
$\$0$		$n+n+\$$	Shift
$\$02$	n	$+n+n\$$	Reduce $E \rightarrow n$
$\$01$	E	$+n+n\$$	shift
$\$013$	$E+$	$n+n\$$	shift
$\$0134$	$E+n$	$+n\$$	Reduce $E \rightarrow E+n$
$\$01$	E	$+n\$$	shift
$\$013$	$E+$	$n\$$	shift
$\$01/4$	$E+n$	$\$$	Reduce $E \rightarrow E+n$
$\$01$	E	$\$$	accept

Ex:

$A \rightarrow (A) | a$

A:

step 1: Augment the grammar

$$1) A' \rightarrow A$$

$$2) A \rightarrow (A)$$

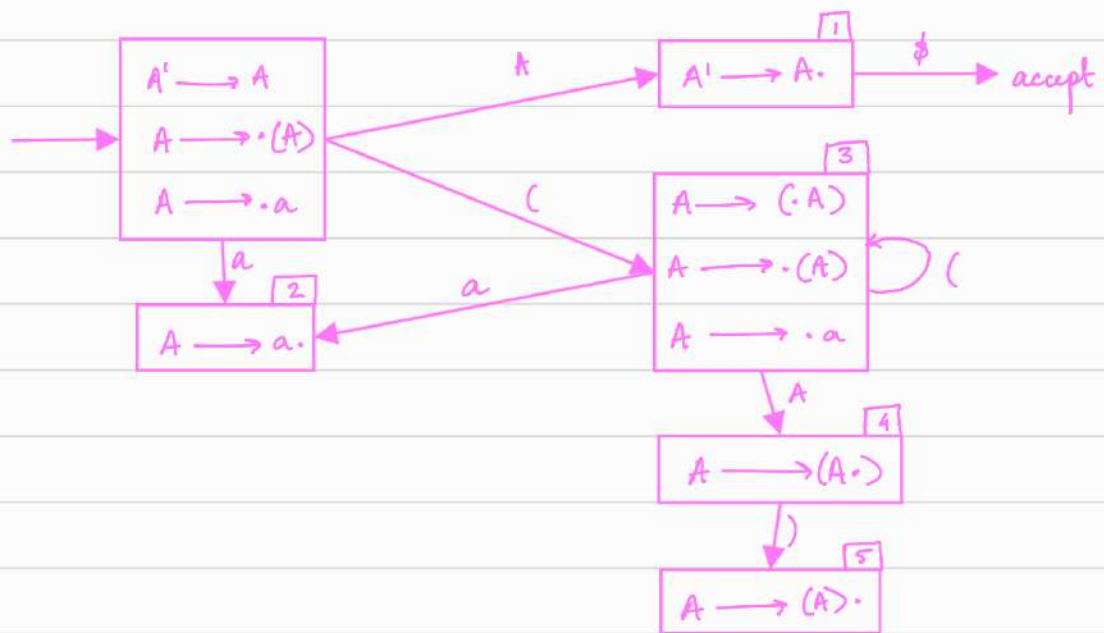
$$3) A \rightarrow a$$

step 2: Start state of LR(0) Automata

$$\text{closure}(A' \rightarrow \cdot A) = \{ A' \rightarrow \cdot A$$

$$A \rightarrow \cdot (A)$$

$$A \rightarrow \cdot a \}$$



LR(0) Automata

follow(A) = {), \$ }

Input = (a)(a)\$

LR(1) base Table

	Action			Goto	
	(	)	a	\$	A
0	S3		S2		1
1				accept	
2		r2		r2	
3	S3		S2		4
4		S5			
5		r1		r1	

Ex:

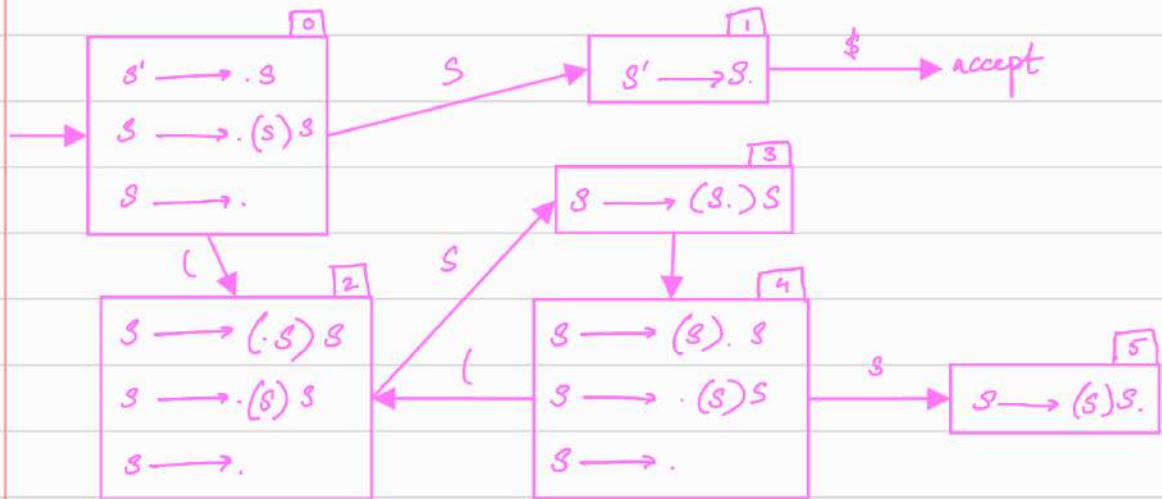
$$S \rightarrow (S)S \mid \epsilon$$

A:

$$0) S' \rightarrow S$$

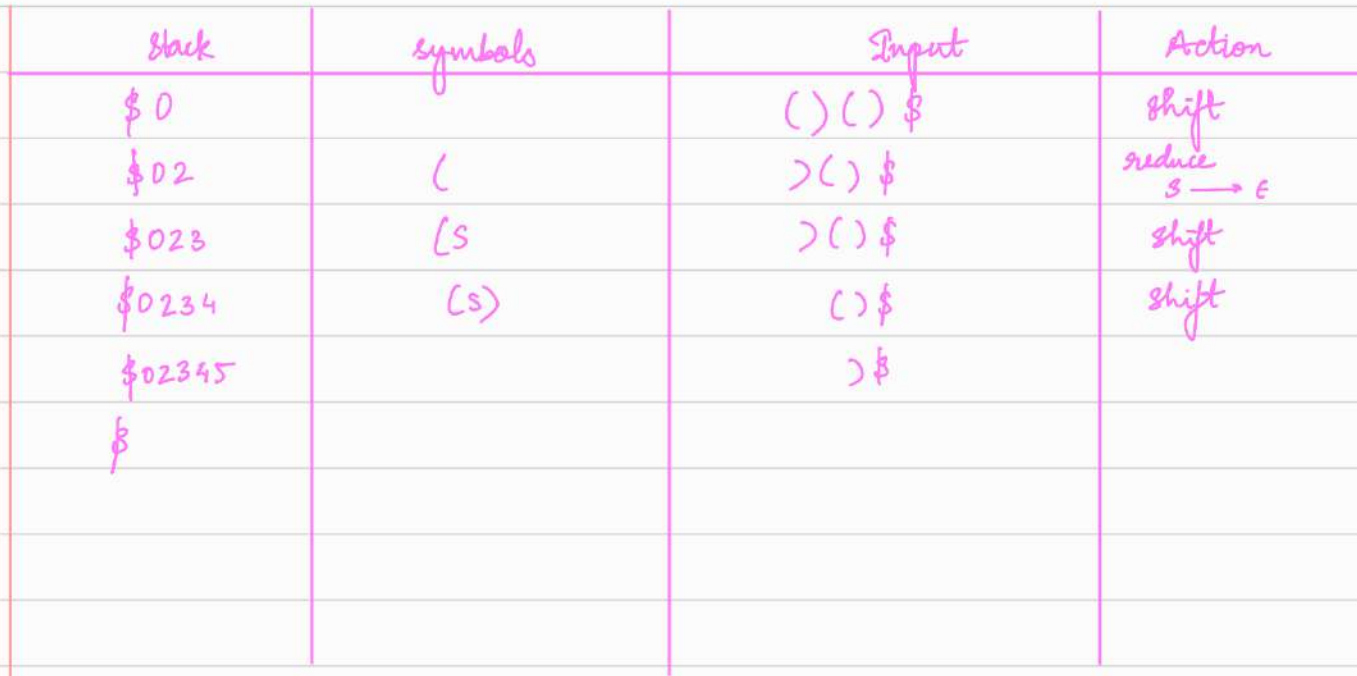
$$1) S \rightarrow (S)S$$

$$2) S \rightarrow \epsilon$$



LR(0) base Table

	Action			Goto
	$($	$)$	$\$$	
0	s2	r2	r2	1
1			Accept	
2	s2	r2	r2	3
3		s4		
4	s2	r2	r2	5
5		r1	r1	



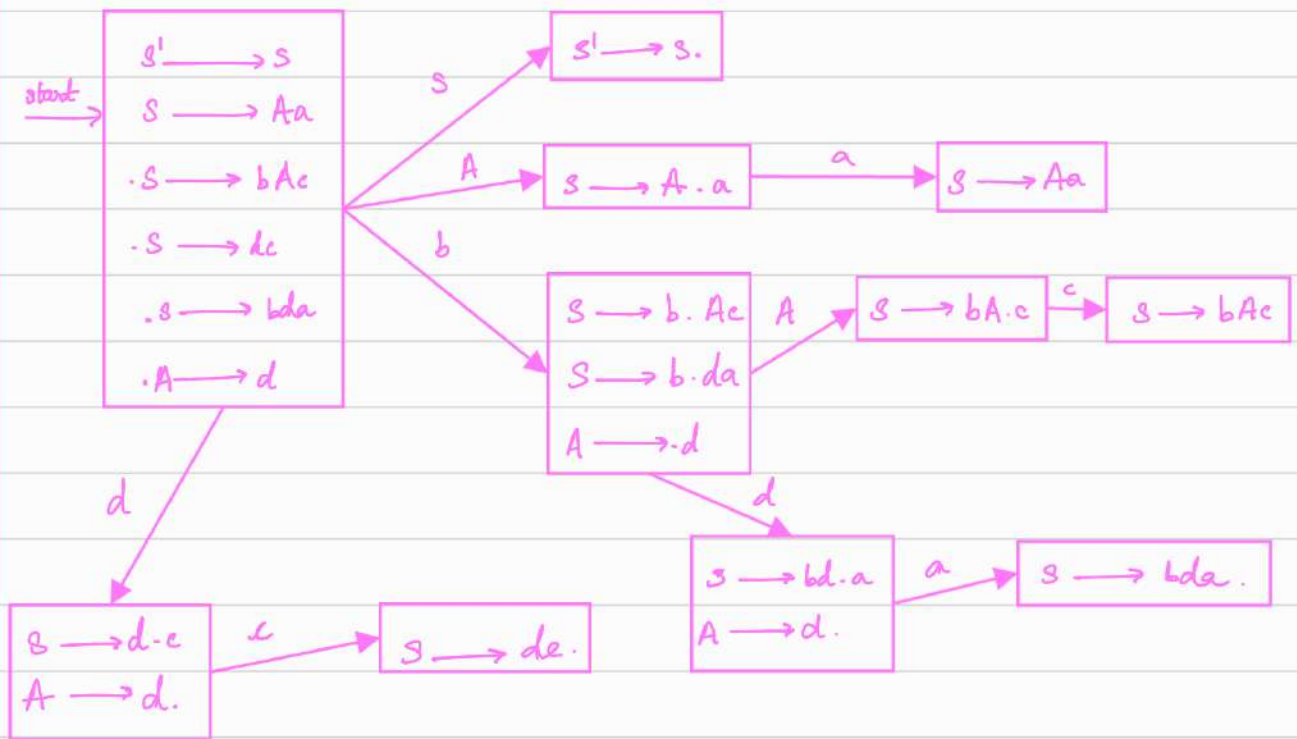
While constructing DFA [LR(0) Auto.] we are not using first & follow

$$\begin{aligned} S &\rightarrow Aa \mid bAc \mid dc \mid bda \\ A &\rightarrow d \end{aligned}$$

closure $(S' \rightarrow S) : \{ \mid S' \rightarrow S$

- 2) $S \longrightarrow Aa$
- 3) $S \longrightarrow bAc$
- 4) $S \longrightarrow de$
- 5) $S \longrightarrow bda$
- 6) $A \longrightarrow d$

LR(0) Automata/DFA



SHR(1)

Since there are shift & Reduce Entries in a cell, the given grammar is not SLR(1).

SLR Parser's are more powerful than shift Reduce, as they cannot eliminate but only reduce-reduce / shift-reduce conflicts

* SLR(0) uses LR(1) which do not use first & follow in Automata

→ Limitations of SLR(1) parsers

- ▷ Applies lookahead after the construction of the DFA of LR(0) items
- ▷ The construction of DFA ignores lookaheads

Q) $S \rightarrow L = R / R$
 $L \rightarrow * R / id$
 $R \rightarrow h$

A:

1) $S' \rightarrow S$
2) $S \rightarrow L = R / R$
3) $L \rightarrow * R / id$
4) $R \rightarrow h$

} S-R

5) $A \rightarrow SA / a$
6) $S \rightarrow AS / b$

} R-R

$S' \rightarrow S$
 $S \rightarrow L = R \mid R$
 $S \rightarrow R$
 $L \rightarrow *R$
 $L \rightarrow id$
 $R \rightarrow L$

$S \rightarrow S.$

Accept

$S \rightarrow L. = R$

$S \rightarrow L = . R$

$R \rightarrow L.$

$R \rightarrow . L$

CD lab 10: YACC [now known as yacc]

→ tokens are non-terminals

→ $x \rightarrow =, ^\wedge$; left $\rightarrow -, +, *, /$

$\hookrightarrow = < - + < * / < ^\wedge$

* final end sem
6/4/26

* 2 eval
on 26/3/26

flex & bison

4.

input $\rightarrow$ input line $| \epsilon$

input $\rightarrow \epsilon$ input'

input' $\rightarrow$ line input' $| \epsilon$

line $\rightarrow$ '\n' $|$ exp '\n'

exp $\rightarrow$ num $|$ exp exp '+'

$|$ exp exp '-'

$|$ exp exp '*'

$|$ exp exp '/'

$|$ exp exp '^'

$|$ exp 'n'

$A \rightarrow A\alpha | \beta$

$\Rightarrow A \rightarrow \beta A'$

$\Rightarrow A' \rightarrow \alpha A' | \epsilon$

26/3/26

Canonical LR(1) Parser

Ex:

$$A \rightarrow (A) | a$$

A:

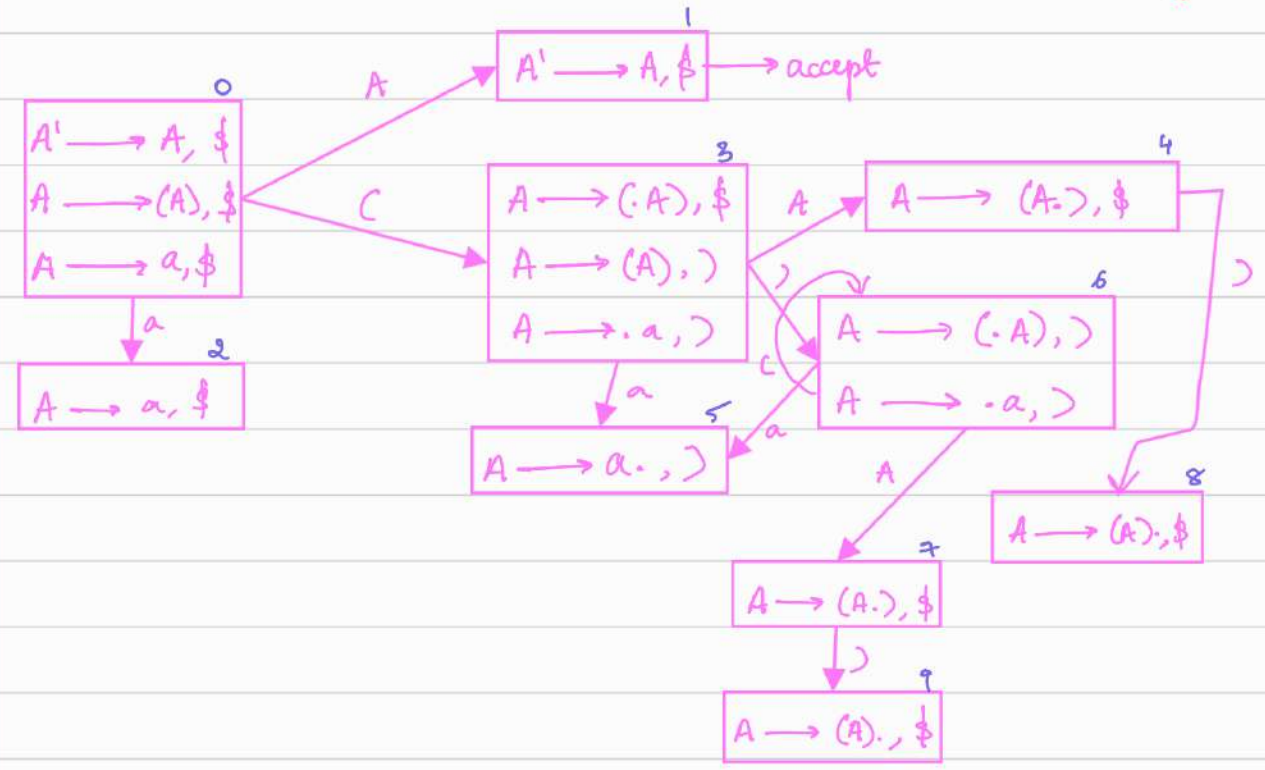
$A' \rightarrow A$	first	follow
$A \rightarrow (A)$	{(, a}	{), \$}
$A \rightarrow a$		

* LR(1) items has 2 parts
 ↳ first LR(0) items
 ↳ lookahead

* follow of start symbol always contain (.

closure $\{A' \rightarrow A, \$\} = \{A' \rightarrow \cdot A, \$$
 $A \rightarrow \cdot (A), \$$
 $A \rightarrow \cdot a, \$$
 $\}$

* NOTE:- first items in each state carries the lookahead in every state.



	a	L	)	\$	A
0	s2	s3			1
1				accept	
2				s2	
3	s5	s6			s4
4			s9		
5			s2		
6	s5	s6			s7
7			s8		
8			s4		
9				s4	

Q) $S \rightarrow CC$
 $C \rightarrow cC \mid d$

A:

- 1) $S' \rightarrow S$
- 2) $S \rightarrow CC$
- 3) $C \rightarrow cC$
- 4) $C \rightarrow d$

start

$S' \rightarrow \cdot S, \$$
$S \rightarrow \cdot CC, \$$
$C \rightarrow \cdot cC, c \mid d$
$C \rightarrow \cdot d, c \mid d$

$S' \rightarrow S \cdot, \$$
$S \rightarrow C \cdot C, \$$
$C \rightarrow \cdot cC, \$$
$C \rightarrow \cdot d, \$$

$S \rightarrow CC \cdot, \$$

$C \rightarrow c \cdot C, \$$
$C \rightarrow$
$C \rightarrow d \mid \$$

$C \rightarrow d \cdot, c \mid d$

$C \rightarrow c \cdot C, c \mid d$
$C \rightarrow \cdot cC, c \mid d$
$C \rightarrow \cdot d, c \mid d$

$C \rightarrow d \cdot, \$$

$C \rightarrow cC \cdot, c \mid d$

$C \rightarrow cC \cdot, \$$

0					
1					
2					
3					
4					
5					
6					
7					
8					
9					

6)

Construct LR(1)

$$S \rightarrow L = R / R$$

$$L \rightarrow * R \mid id$$

$$R \rightarrow L$$

} 14 states

1:

1) $S' \rightarrow S$

2) $S \rightarrow L = R / R$

3) $L \rightarrow * R \mid id$

4) $R \rightarrow L$

7 shift entries
0-2, 4-5,
4 states - id, *

start



$S' \rightarrow \cdot S, \$$
$S \rightarrow \cdot L = R, \$$
$S \rightarrow \cdot R, \$$
$L \rightarrow \cdot * R, = \$$
$L \rightarrow \cdot id, = \$$
$R \rightarrow \cdot L, \$$

$S' \rightarrow S \cdot, \$$

$S \rightarrow L \cdot = R, \$$
$S \rightarrow R \cdot, \$$

$L \rightarrow * \cdot R, \$$
$L \rightarrow id \cdot, \$$

$$L \rightarrow \cdot id, = | \$$$

$$L \rightarrow \cdot * R, = | \$$$

19/3/26
→

SLR(1)

Q) $E \rightarrow E+n/n$

A:

Step 1: Augment the grammar

$E' \rightarrow E$

$E \rightarrow E+n$

$E \rightarrow n$

Step 2: No. from zero

1) $E' \rightarrow E$

2) $E \rightarrow E+n$

3) $E \rightarrow n$

Step 3: Build LR(0) Automata (DFA)

Step 4: Finding the start state of DFA

obtained by,

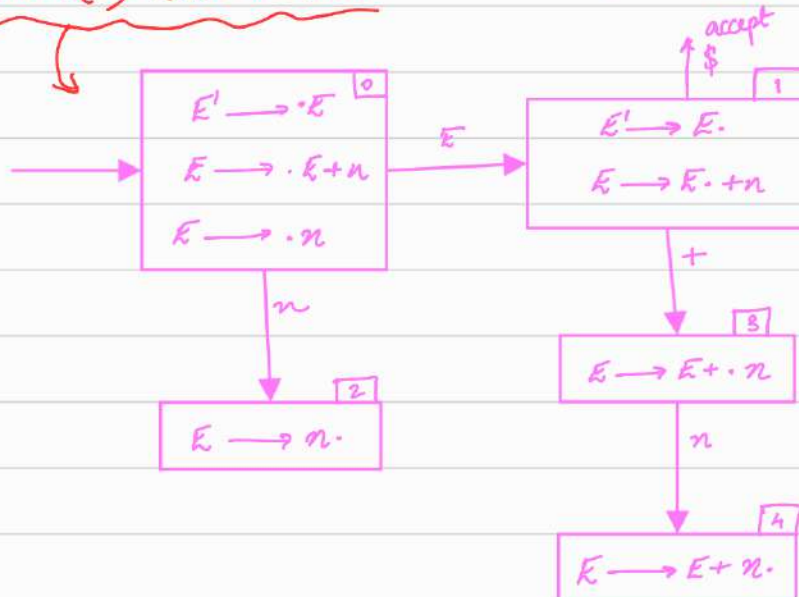
closure ($E' \rightarrow \cdot E$)

$\{ E' \rightarrow \cdot E \}$ non-terminal

$E \rightarrow \cdot E+n$

$E \rightarrow \cdot n$

Step 3: LR(0) Automata DFA



NOTE:- Row headers are states no's. & col headers are symbols (terminals & nonterminals)

SLR(1) Parse Table

	Action			Goto	
	n	$+$	$\$$	E'	E
0	S_2				1
1		S_3	accept		
2		r_2	r_2		
3	S_4				
4		r_1	r_1		

★ S_2 for instance, here $S \rightarrow$ shift & no. associated $\rightarrow$ state no.
 whereas reduce $\rightarrow$ prod. rule no.

★ start sym.
 for stack is
 $\$$ & start
 state

pop 3 sym.
 from
 stack

Stack	symbols	Input	Action
$\$ 0$		$n+n+\$$	Shift
$\$ 02$	n	$+n+n\$$	Reduce $E \rightarrow n$
$\$ 01$	E	$+n+n\$$	shift
$\$ 013$	$E+$	$n+n\$$	shift
$\$ 0134$	$E+n$	$+n\$$	Reduce $E \rightarrow E+n$
$\$ 01$	E	$+n\$$	shift
$\$ 013$	$E+$	$n\$$	shift
$\$ 01/4$	$E+n$	$\$$	Reduce $E \rightarrow E+n$
$\$ 01$	E	$\$$	accept

Ex:

$A \rightarrow (A) \mid a$

A:

step 1: Augment the grammar

$$1) A' \rightarrow A$$

$$2) A \rightarrow (A)$$

$$3) A \rightarrow a$$

step 2: Start state of LR(0) Automata

$$\text{closure}(A' \rightarrow \cdot A) = \{ A' \rightarrow \cdot A$$

$$A \rightarrow \cdot (A)$$

$$A \rightarrow \cdot a \}$$

Ex:

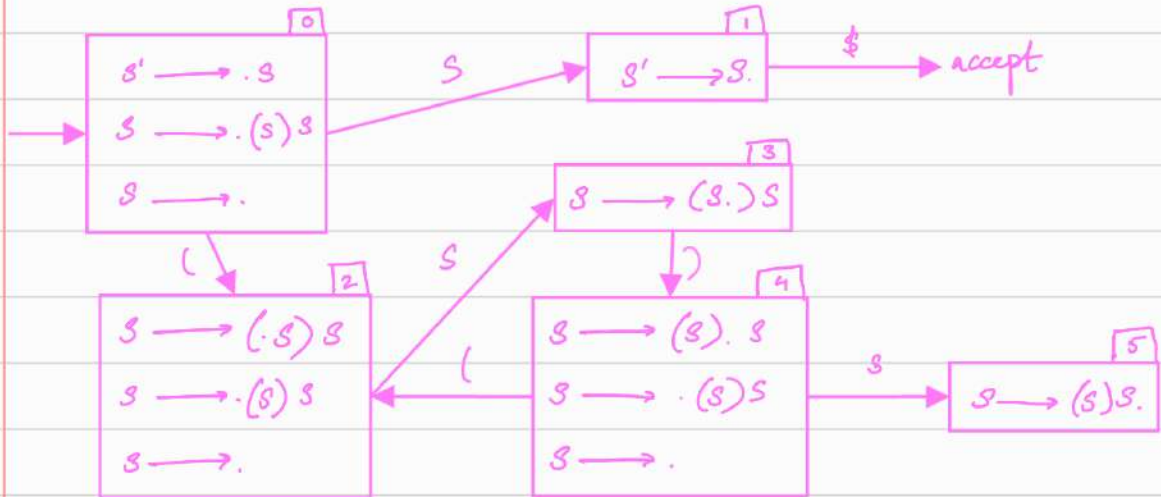
$$S \rightarrow (S)S \mid \epsilon$$

A:

$$0) S' \rightarrow S$$

$$1) S \rightarrow (S)S$$

$$2) S \rightarrow \epsilon$$



LR(0) base Table

	Action			Goto
	$($	$)$	$\$$	
0	s2	r2	r2	1
1			Accept	
2	s2	r2	r2	3
3		s4		
4	s2	r2	r2	5
5		r1	r1	

SHR(1)

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

1) $E' \rightarrow E$

2) $E \rightarrow E+T$

3) $E \rightarrow T$

4) $T \rightarrow T * F$

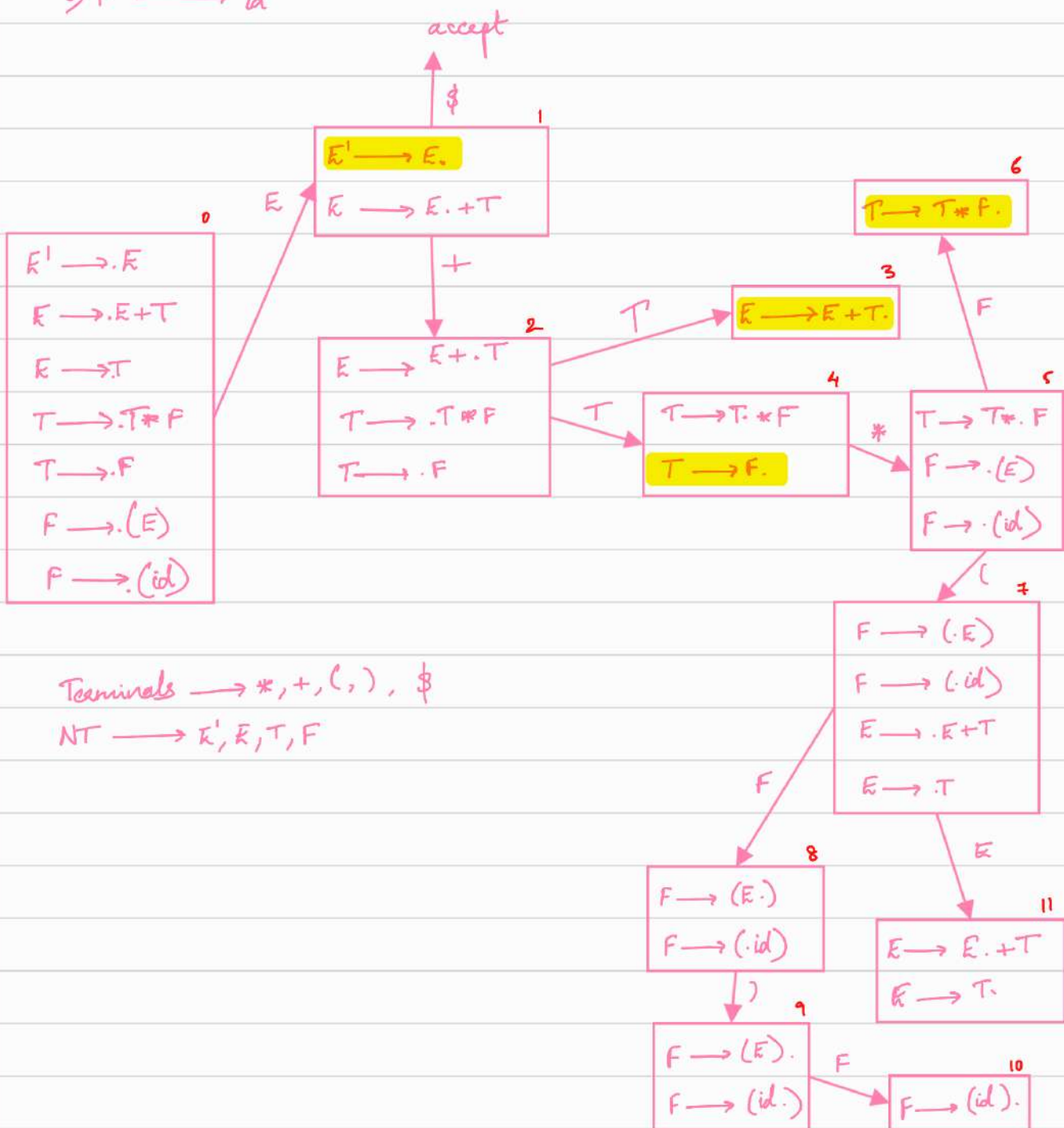
5) $T \rightarrow F$

6) $F \rightarrow (E)$

7) $F \rightarrow id$

Sym $\rightarrow$ getSymbol
*, (,), +, \$

Action [] []
Goto [] []
Production []
Production head []
Production len []



Terminals $\rightarrow *, +, (,), \$$

NT $\rightarrow E', E, T, F$

SLR(1) Parse Table

[illegible]

M3 : Semantic Analysis

Syntax-directed translation

→ refers to a method of compiler implementation where the source

→ done during

→ & notations used for,

① SDD (Syntax Directed Definition) (what kind of action needs to be taken)

② SDT (Syntax Directed Translation)

① SDD (CFG followed by an attribute)

• Attribute

• each prod. in a grammar, we give semantic rules

• Eg - Production

$$E \rightarrow E + T$$

Semantic Rule

$$E.\text{code} = E.t.\text{code} \parallel T.\text{code} \parallel '+'$$

attribute

② SDT

• embeds prog. fragments called semantic actions within prod. rules

NOTE: - multiple rules for same prod. is poss. in SDD.

Attributes

① Synthesized [up the parse tree, Bottom-up]

• the lexical analyzer usually supplies the attributes of terminals and the synthesized ones are built up for the NT & passed up the tree.

N is a node in parse tree

* synthesized attribute at node N is defined only in terms of attribute values at the children of N and at N itself.

② Inherited [Top-down parsing]

* inherited attribute at node N is defined only in terms of att. values at the parent node N itself or its siblings

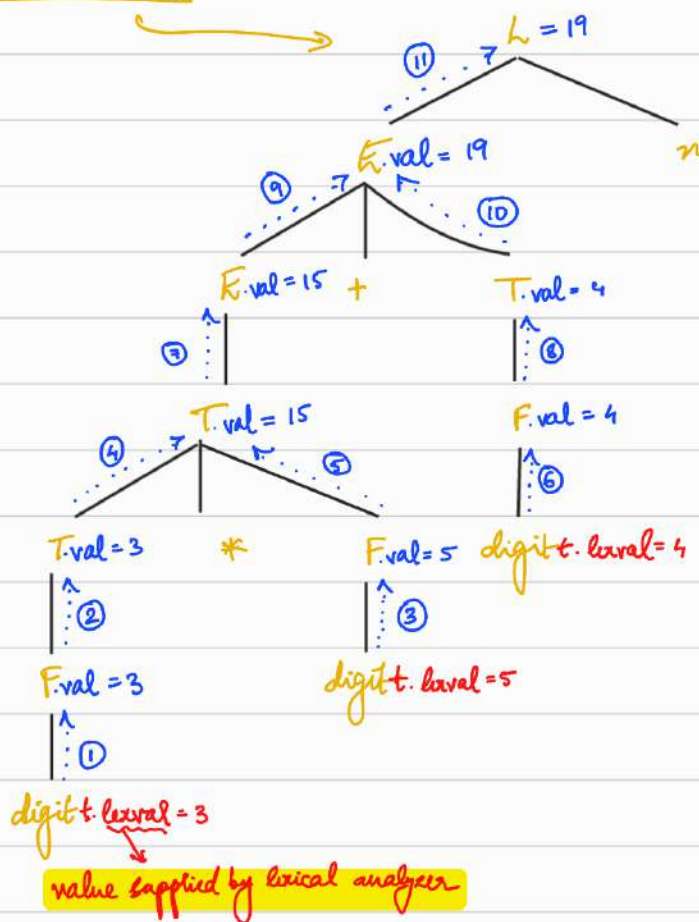
Q) SDD Question

- 1) $L \rightarrow E$ instead of $\$$; i/p string: - 3 * 5 + 4
- 2) $E \rightarrow E_1 + T$
- 3) $E \rightarrow T$
- 4) $T \rightarrow T_1 * F$
- 5) $T \rightarrow F$
- 6) $F \rightarrow (E)$
- 7) $F \rightarrow \text{digit}$

Purpose: to build arithmetic calculator

It is already augmented $\rightarrow$ top-down $\rightarrow$ inherited attribute

Annotated Parse Tree (top-down)



* order of evaluation is using dotted lines from LHS $\rightarrow$ RHS

Dependency edges
1, 6, 11 $\Rightarrow$ evaluation no's

continuation
(How to write rules)

* NOTE:- there's no requirement for '()' as such in semantic analysis except in parse tree for precedence.

- 1) $L \rightarrow E \text{ or}$
- 2) $E \rightarrow E_1 + T$
- 3) $E \rightarrow T$
- 4) $T \rightarrow T_1 * F$
- 5) $T \rightarrow F$
- 6) $F \rightarrow (E)$
- 7) $F \rightarrow \text{digit}$

$$\left\{ \begin{array}{l} L.\text{val} = E.\text{val} \\ E.\text{val} = E_1.\text{val} + T.\text{val} \\ E.\text{val} = T.\text{val} \\ T.\text{val} = T_1.\text{val} * F.\text{val} \\ T.\text{val} = F.\text{val} \\ \text{there's no production} \\ F.\text{val} = \text{digit lexical} \end{array} \right.$$

Rules writing :- [starts from below]

7/4/26 →

Inherited Attributes

$D \rightarrow Th$ $\{h.in := T.type;\}$

$T \rightarrow int$ $\{T.type := integer;\}$

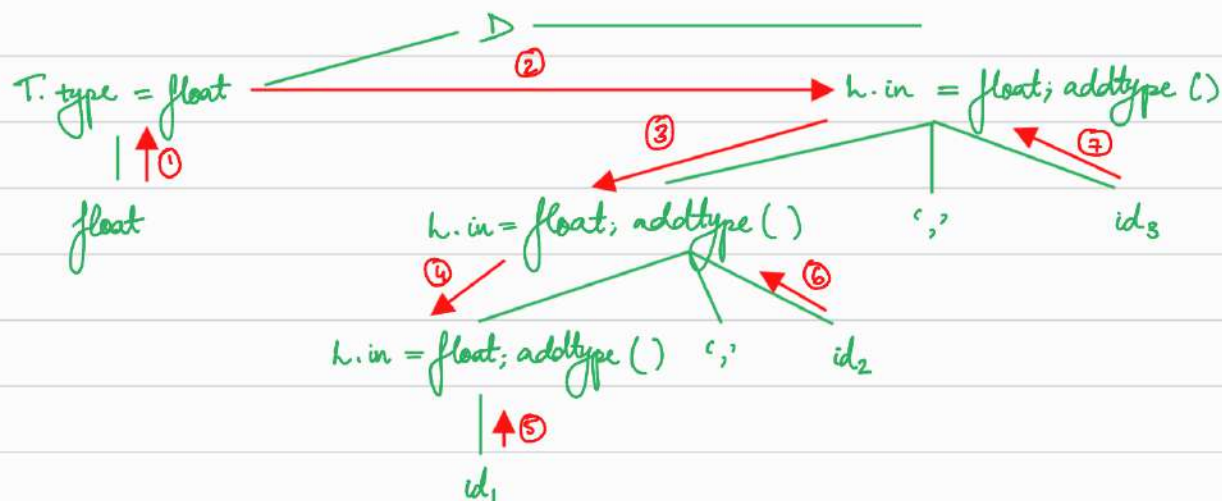
$T \rightarrow float$ $\{T.type := float;\}$

$h \rightarrow$ $\{h.in := h.in;\}$

$h_1 \text{ ',' } id$ $\{addtype(id.entry, h.in);\}$

$h \rightarrow id$ $\{addtype(id.entry, h.in);\}$

[addtype makes/ adds value of h.in in symbol table for id.in] NOTE



→

Application of SDT

→ Purpose: construct syntax tree (aka abstract syntax tree)

→ interior nodes $\Rightarrow$ operators

leaves $\Rightarrow$ identifiers/numbers

$E \rightarrow E + T$

$E.node = \text{new Node}('+', E_1.node, T.node)$

$E \rightarrow E_1 - T$

$E.node = \text{new Node}('-', E_1.node, T.node)$

$E \rightarrow T$

$E.node = T.node$

$T \rightarrow (E)$

$T.node = E.node$

$T \rightarrow id$

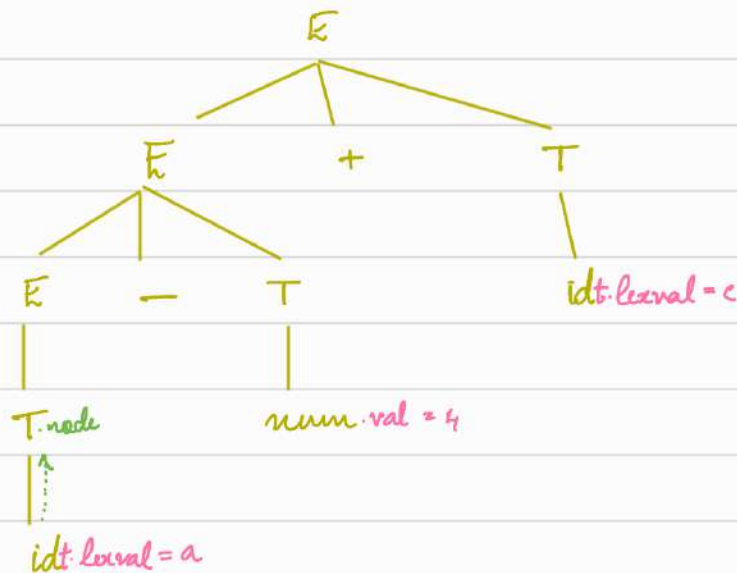
$T.node = \text{new leaf}(id, id, entry)$

$T \rightarrow num$

$T.node = \text{new leaf}(num, num, val)$

Q) $a - 4 + c$

A:



→ MH: Intermediate Code Generator

→ Need for ICG

→ Intermediate Expression

- Postfix Notation
- Abstract Syntax tree
- Three Address Code

→ Variant of Syntax Tree

→ DAG (Directed Acyclic Graph)

A node N in a DAG has more than one parent node if node N express more than one common expression

eg:- $K = K - 7$



fig:- Syntax tree

⇒



fig:- DAG

Don't have redundant nodes

Q) $a + b + (a + b)$

* when op. have same precedence it's $\rightarrow R$

$a + b + (a + b)$
② ③ ①

A:

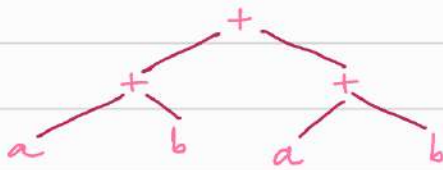


fig:- Syntax tree

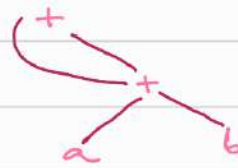


fig:- DAG

Q) $a + b + a + b$

A:

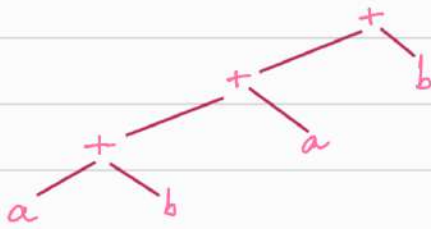


fig:- Syntax tree

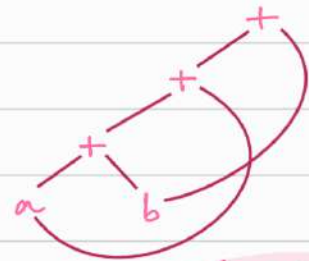


fig:- DAG

Q4/26

Q) $a + a + (a + a + a + (a + a + a + a))$

A:

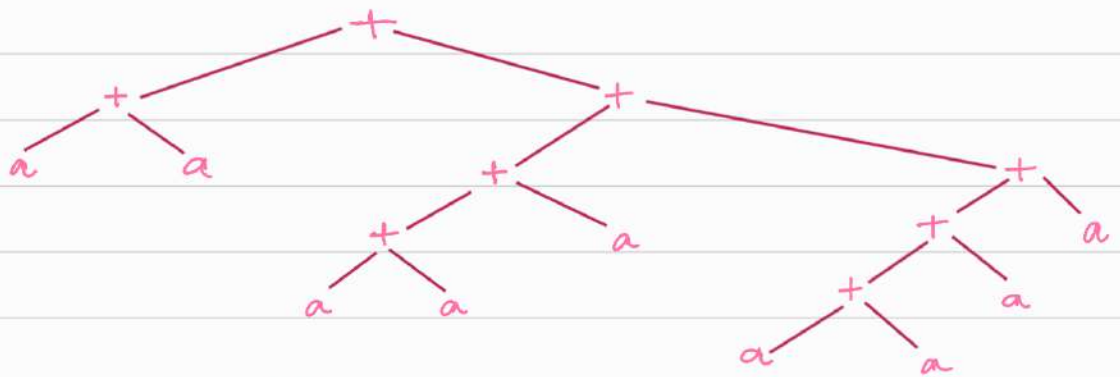
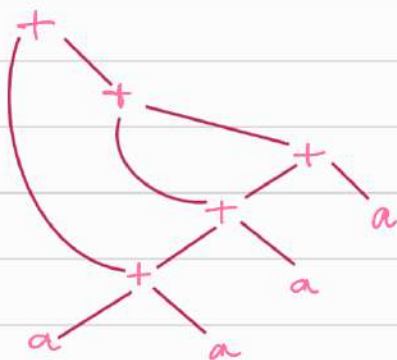
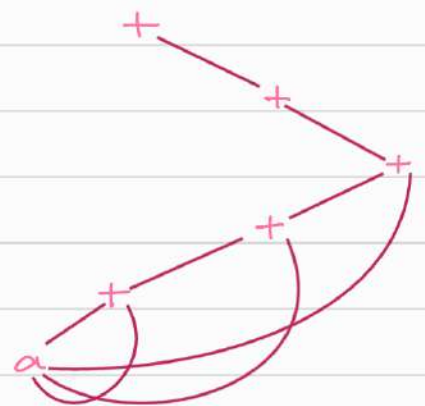


fig:- Syntax tree (↑)

fig:- DAG (↓)



redundant
 $\Rightarrow$
correct solution



Q) $((x+y) - ((x+y) * (x-y))) + ((x+y) * (x-y))$

A:

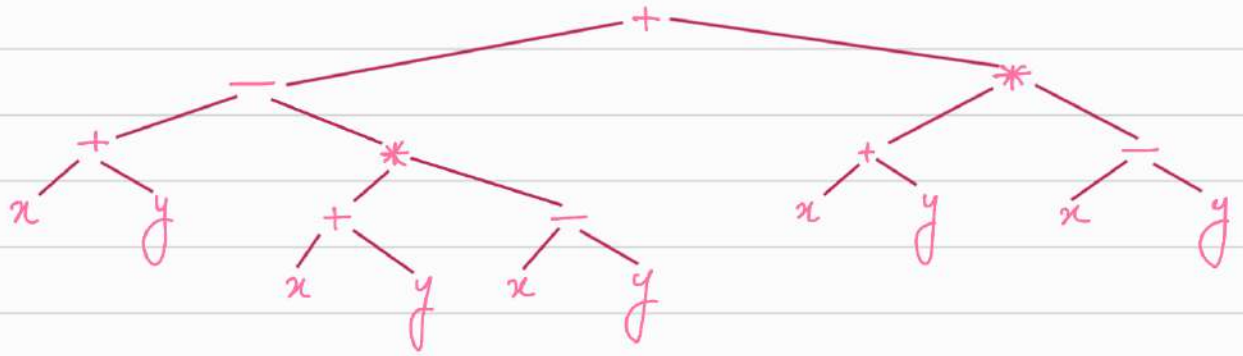
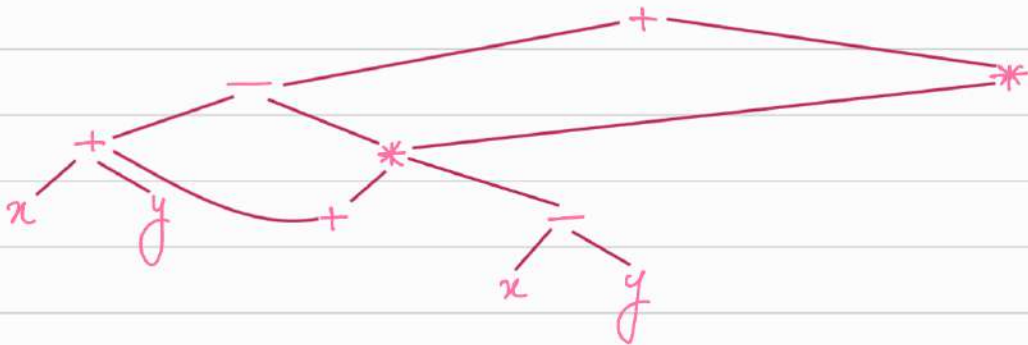


fig:- Syntax tree ($\uparrow$)

fig:- DAG ($\downarrow$)



Q) $(a+b * c) + d + (a+b * c) - d + e$

A:

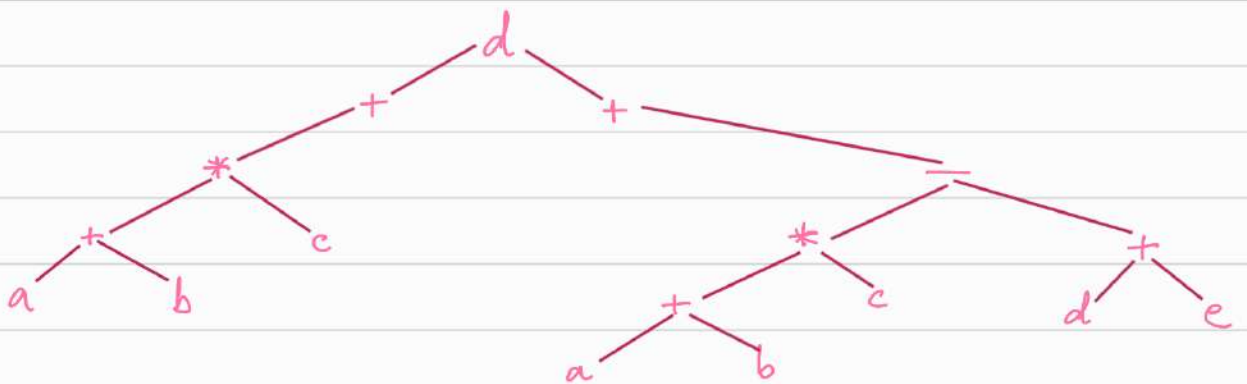


fig:- Syntax tree ($\uparrow$)

fig:- DAG ($\rightarrow$)



(not sure if DAG correct)

H.W
Q)

$$a + a * (b - c) + (b - c) * d$$

A:

→ Implementation of DAG

→ SDD (refer to application of SDD)

→ Value no. Method (inexpensive so we prefer Hashing)
already done before

→ Value Number Method

* NOTE:- ① $r = 0$ only used for unary operators

② always starts from the first index

Fig:- $i = i + 10$



	operator	l	r
1	num	10	
2	id		entry to i
3	—	1	0
4	+	2	3
5	=	2	4

⟶ $\langle op, l, r \rangle$

Three address code

C statement	TAC
$Count = index * 20$	$t1 = index * 20$ $Count = t1$
$p_{new} = p + ((p * n * r) / 100)$	$t1 = p * n$ $t2 = t1 * r$ $t3 = t2 / 100$ $t4 = p + t3$ $p_{new} = t4$
$a = -b$ $a = b$	only for copy / unary you can directly assign $a = -b$ $a = b$
unconditional jump	go to L
conditional jump if($x < y$) S1 else S2	if($x < y$) go to L1 go to L2 L1 S1 go to End L2 S2 End
Indexed assignment $y = x[i]$	$t1 = i * 8$ $t2 = x[t1]$ $y = t2$
Address assignment	$x = \&y$
Pointer Assignments	$y = \&x$ $x = *y$

Q

$$a = b[i] + c$$

A:

$$t1 = i * 8$$

$$t2 = b[t1]$$

$$t3 = t2 + c$$

$$a = t3$$

Now implementation of TAC.....

→

Implementation of TAC

- ① Quadraple (opcode, arg1, arg2, Result)
- ② Triplus (opcode, arg1, arg2, (here result is found using positions))
- ③ Indirect Triplus

NOTE: goto is not a valid arg.
goto is JMP

TAC statement		Quadraple			
		opcode	arg1	arg2	Result
1.	$t1 = y + z$	+	y	z	t1
2.	$x = -y$	-	y		x
3.	$x = y$	=	y		x
4.	goto to h	JMP			h
5.	if ($x < y$) goto h1	JLT	x	y	h1
	$x = y[i]$	[]	y	i	x
	$x = *y$	*	y		x

$$x. \quad y[i] = x$$

A:

opcode	arg1	arg2	Result
[]	y	i	t1
=	x		t2

Q) $a = b[i] + c$

A:

$$t1 = i * 8$$

$$t2 = b[t1]$$

$$t3 = t2 + c$$

$$a = t3$$

for
Quadruples

TAC statement		Quadruple			
		opcode	arg1	arg2	Result
$t1 = i * 8$		*	i	8	t1
$t2 = b[t1]$		[]	b	t1	t2
$t3 = t2 + c$		+	t2	c	t3
$a = t3$		=	t3		a
	pos	opcode	arg1	arg2	
	0	*	i	8	for triples
	1	[]	b	(0)	
	2	+	(1)	c	
		=	a	(2)	

Q) Write Quadraple & triple for statement.

$$a = b * -c + b * -c$$

A: $t1 = -c$ $t5 = t3 + t4$
 $t2 = -c$ $a = t5$
 $t3 = b * t1$
 $t4 = b * t2$

TAC statement	Quadraple			
	opcode	arg1	arg2	Result
$t1 = -c$	-		c	t1
$t2 = -c$	-		c	t2
$t3 = b * t1$	*	b	t1	t3
$t4 = b * t2$	*	b	t2	t4
$t5 = t3 + t4$	+	t3	t4	t5
$a = t5$	=		t5	a

pos	opcode	arg1	arg2	
0	-		c	
1	-		c	
2	*	b	(0)	
3	*	b	(1)	
4	+	(2)	(3)	
	=	a	(4)	

Q)

do

$i = i + 1;$

while ($a[i] < v$);

Ans:

L1: $t1 = i + 1$

$i = t1$

$t2 = i * 8$

$t3 = a[t2]$

if $t3 < v$ goto L1
goto L2

L2:

size of 'i' not mentioned so assumed '8'

TAC statement		Quadruple			
		opcode	arg1	arg2	Result
$t1 = i + 1$	L1:	+	i	1	t1
		=	t1		i
		*	i	8	t2
		[]	a	t2	t3
		JLT	t3	v	L1
		JMP			L2
	L2:				
	pos	opcode	arg1	arg2	
	0	+	i	1	
	1	=	(0)		
	2	*	i	8	
	3	[]	a	(2)	
	4	JLT	(3)	v	
	5	JMP			
	6				